

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Giang Văn Huy

**NGHIÊN CỨU VÀ PHÁT TRIỂN HỆ THỐNG
KIỂM SOÁT CHẤT LƯỢNG KHÔNG KHÍ
TRONG CĂN HỘ CHUNG CƯ ỨNG DỤNG
CÂY TRỒNG TRONG NHÀ**

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Kỹ thuật máy tính

**Cán bộ hướng dẫn: TS. Hoàng Gia Hưng
PGS.TS. Phạm Minh Triển**

HÀ NỘI - 2026

TÓM TẮT

Tóm tắt: Khóa luận này trình bày các kết quả nghiên cứu và quy trình thiết kế một hệ thống giám sát và điều khiển chất lượng không khí trong nhà (Indoor Air Quality - IAQ) sử dụng cây trồng thủy sinh, được thiết kế chuyên biệt cho không gian kín như căn hộ chung cư. Nghiên cứu có sự kết hợp giữa nền tảng Internet vạn vật (IoT) và các giải pháp thực sinh học, cụ thể là ứng dụng cây lan ý (*Spathiphyllum*). Khác với các hướng tiếp cận truyền thống vốn chỉ xem cây như một màng lọc khí thụ động, nghiên cứu này coi thực vật là một hệ thống sinh học tự động có khả năng tương tác. Khả năng này chỉ được phát huy tối đa khi môi trường sinh trưởng được duy trì ở trạng thái lý tưởng. Kế thừa các công bố khoa học về ứng dụng thủy canh trong kiểm soát IAQ, lan ý đã được chứng minh là loài thực vật sở hữu năng lực vượt trội trong việc đồng hóa CO₂ và hấp thụ các hợp chất hữu cơ dễ bay hơi (VOC) tích tụ trong không gian kín [15, 8]. Phát triển từ đó, mục tiêu cốt lõi của đề tài là tạo một hệ sinh thái khép kín: vừa giám sát, cảnh báo tự động, vừa điều phối các điều kiện vi khí hậu nhằm tối ưu hóa sự phát triển của thực vật và bảo vệ sức khỏe con người.

Về mặt kiến trúc hệ thống, nghiên cứu triển khai mô hình phân lớp chuẩn mực bao gồm: sensor node, gateway, cloud và application. Trong đó, node sử dụng vi điều khiển Arduino Pro Mini để đo lường các tham số môi trường và sinh trưởng như: chỉ số tổng chất rắn hòa tan (TDS), độ pH, nhiệt độ và độ ẩm không khí, nồng độ CO₂ và cường độ ánh sáng. Nhằm khắc phục triệt để nhược điểm về vùng phủ sóng của mạng Wi-Fi truyền thống trong cấu trúc căn hộ nhiều vật cản, hệ thống đã ứng dụng công nghệ truyền thông công suất thấp, khoảng cách xa LoRa để kết nối node với gateway. Tại lớp trung gian này, gateway không chỉ đóng vai trò chuyển tiếp mà còn thực thi tiền xử lý, kiểm định tính toàn vẹn của dữ liệu trước khi đồng bộ lên nền tảng đám mây ThingsBoard. Thingsboard cũng cung cấp giao thức giao tiếp ứng dụng (API) cho phần mềm di động được phát triển trên Flutter. Từ đó, hệ thống trao quyền cho người dùng khả năng giám sát theo

thời gian thực và trực tiếp can thiệp vào các thiết bị ngoại vi (quạt thông gió, đèn chiếu sáng, điều hòa không khí) thông qua rơ-le và sóng hồng ngoại.

Những đóng góp khoa học và thực tiễn của nghiên cứu tập trung vào việc giải quyết bài toán tích hợp hệ thống: từ thiết kế phần cứng, chuẩn hóa giao thức truyền thông, kiến trúc dữ liệu trên ThingsBoard cho đến việc thiết kế application . Để đảm bảo sự sinh trưởng bền vững của hệ thủy canh trong không gian hẹp, hệ thống sử dụng các ngưỡng giới hạn nghiêm ngặt về pH (6.0 – 6.5) và độ dẫn điện (EC) ở mức xấp xỉ 1.2 dS/m(768ppm), dựa trên các tiêu chuẩn nông nghiệp hiện hành [17, 16]. Các chuỗi thực nghiệm đánh giá đã khẳng định độ tin cậy và khả năng đáp ứng yêu cầu của hệ thống trước những biến động bất lợi của môi trường, điển hình như sự gia tăng đột biến của CO₂, thiếu hụt ánh sáng hay sự mất cân bằng chất dinh dưỡng. Kết quả này không chỉ mang tính ứng dụng cao mà còn tạo tiền đề vững chắc cho các nghiên cứu tiếp theo về ứng dụng trí tuệ nhân tạo trong kiểm soát không gian sống.

Từ khóa: *Chất lượng không khí trong nhà (IAQ), Internet vạn vật (IoT), lan ý, Spathiphyllum, hệ thủy canh, Arduino Pro Mini, ESP32, truyền thông LoRa, ThingsBoard, Flutter, nồng độ CO₂, điều khiển thích nghi.*

LỜI CẢM ƠN

Đầu tiên, em xin gửi lời cảm ơn chân thành đến toàn thể quý thầy, cô trong Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội, đặc biệt là các thầy, cô công tác tại khoa Điện tử - Viễn thông, vì đã tạo điều kiện và hỗ trợ em trong quá trình học tập, nghiên cứu.

Em xin gửi lời cảm ơn sâu sắc đến cán bộ hướng dẫn đã định hướng, góp ý và hỗ trợ em trong quá trình thực hiện khóa luận. Những nhận xét và chỉ dẫn của thầy là cơ sở quan trọng giúp em hoàn thiện đề tài cả về nội dung chuyên môn lẫn cách trình bày.

Cuối cùng, em xin cảm ơn gia đình, bạn bè đã luôn đồng hành, động viên và hỗ trợ em trong thời gian học tập cũng như trong quá trình hoàn thành khóa luận này.

Giang Văn Huy

LỜI CAM ĐOAN

Em xin cam đoan khóa luận “Nghiên cứu và phát triển hệ thống kiểm soát chất lượng không khí trong căn hộ chung cư sử dụng cây trồng trong nhà” được thực hiện bởi bản thân dưới sự hướng dẫn của cán bộ hướng dẫn.

Mọi tham khảo, trích dẫn từ các nghiên cứu và tài liệu liên quan được sử dụng trong khóa luận đều được nêu nguồn gốc một cách rõ ràng tại danh mục tài liệu tham khảo. Trong khóa luận hoàn toàn không có sự sao chép tài liệu cũng như công trình nghiên cứu của người khác.

Hà Nội, ngày ... tháng ... năm 2026

Tác giả

Giang Văn Huy

PHÊ DUYỆT CỦA CÁN BỘ HƯỚNG DẪN

Tôi xác nhận rằng quyển khóa luận này của sinh viên Giang Văn Huy đã đáp ứng các yêu cầu để đưa ra bảo vệ trước Hội đồng.

Hà Nội, ngày ... tháng ... năm 2026

Cán bộ hướng dẫn

.....

Mục lục

TÓM TẮT	i
LỜI CẢM ƠN	iii
LỜI CAM ĐOAN	iv
PHÊ DUYỆT CỦA CÁN BỘ HƯỚNG DẪN	v
1 GIỚI THIỆU	1
1.1 Đặt vấn đề	1
1.2 Tổng quan nghiên cứu	3
1.3 Mục tiêu của đề tài	5
1.4 Đối tượng và phạm vi nghiên cứu	6
1.5 Phương pháp nghiên cứu	6
1.6 Ý nghĩa thực tiễn	7
1.7 Bố cục khóa luận	8
2 CƠ SỞ LÝ THUYẾT	9
2.1 Chất lượng không khí trong nhà	9
2.1.1 Nhiệt độ và độ ẩm	10
2.1.2 Nồng độ CO ₂	10
2.1.3 Hợp chất bay hơi và giới hạn của hệ thống	10
2.2 Cây lan ý và các thông số chăm sóc	11

2.2.1	TDS và pH	11
2.2.2	Ánh sáng	12
2.2.3	Liên hệ giữa chăm sóc cây và IAQ	12
2.3	Node cảm biến Arduino Pro Mini	12
2.3.1	Yêu cầu đối với node	13
2.4	Truyền thông LoRa và gateway ESP32	13
2.4.1	Luồng truyền dữ liệu	13
2.5	ThingsBoard và ứng dụng Flutter	14
3	THIẾT KẾ PHẦN CỨNG NODE	15
3.1	Yêu cầu thiết kế phần cứng	15
3.2	Phân rã chức năng theo khối	17
3.3	Nguyên tắc bố trí chân và tín hiệu	18
3.4	Thiết kế khối nguồn	19
3.4.1	Nguồn đầu vào và bảo vệ	19
3.4.2	Mạch hạ áp 5V	19
3.4.3	Mạch ổn áp 3.3V	20
3.4.4	LED báo nguồn	20
3.4.5	Rủi ro và biện pháp giảm nhiễu	21
3.5	Khối vi điều khiển Arduino Pro Mini	21
3.5.1	Vai trò của vi điều khiển	21
3.5.2	Bố trí header	22
3.5.3	Quản lý chân vào ra	22
3.5.4	Chu kỳ hoạt động của node	23
3.5.5	Quy tắc an toàn khi khởi động	23
3.6	Khối giao tiếp LoRa và RS485	24
3.6.1	Module LoRa RA-02	24
3.6.2	Định dạng gói tin LoRa	24

3.6.3	Cơ chế kiểm tra lỗi	25
3.6.4	Mạch MAX485	25
3.6.5	Kiểm thử khối giao tiếp	26
3.7	Khối cảm biến và khối actuator	26
3.7.1	Khối cảm biến	26
3.7.2	Lọc và hiệu chuẩn cảm biến	27
3.7.3	Khối relay	28
3.7.4	Khối phát hồng ngoại	28
3.7.5	Mô hình trạng thái thiết bị	29
3.7.6	Kiểm thử khối cảm biến và actuator	29
4	THIẾT KẾ PHẦN MỀM VÀ TRIỂN KHAI HỆ THỐNG	30
4.1	Kiến trúc tổng thể	31
4.2	Luồng dữ liệu tổng quát	32
4.3	Các chế độ vận hành	32
4.4	Thiết kế node cảm biến	33
4.4.1	Khối đọc cảm biến	33
4.4.2	Khối điều khiển	33
4.4.3	Định dạng gói tin node gửi lên gateway	34
4.4.4	Thuật toán đọc cảm biến	34
4.4.5	Quản lý trạng thái node	35
4.4.6	Xử lý lệnh điều khiển tại node	35
4.5	Thiết kế gateway ESP32 và ThingsBoard	36
4.5.1	Kết nối ThingsBoard	36
4.5.2	Mô hình thiết bị trên ThingsBoard	36
4.5.3	Rule chain và cảnh báo	37
4.5.4	Luồng điều khiển xuống node	37
4.5.5	Xử lý mất kết nối	37

4.6	Thiết kế ứng dụng Flutter	38
4.6.1	Màn hình đăng nhập	39
4.6.2	Màn hình Sensor	39
4.6.3	Màn hình Actuator	39
4.6.4	Ánh xạ dữ liệu từ ThingsBoard sang giao diện	41
4.6.5	Bảo mật và quyền điều khiển	41
5	KẾT QUẢ VÀ THẢO LUẬN	42
5.1	Kịch bản kiểm thử	42
5.2	Kết quả thu thập dữ liệu cảm biến	42
5.3	Kết quả truyền thông LoRa và gateway	43
5.4	Kết quả hiển thị và điều khiển trên ứng dụng	43
5.5	Thảo luận	44
	KẾT LUẬN	45
A	PHỤ LỤC THIẾT KẾ FIRMWARE VÀ GÓI TIN	47
A.1	Mục tiêu của firmware node	47
A.2	Cấu trúc chương trình node	47
A.3	Trình tự khởi động	48
A.4	Vòng lặp chính	49
A.5	Định dạng dữ liệu telemetry	50
A.6	Định dạng gói điều khiển	50
A.7	Gói phản hồi ACK và ERR	51
A.8	Tính checksum	51
A.9	Hiệu chuẩn pH	51
A.10	Hiệu chuẩn TDS	52
A.11	Xử lý dữ liệu CO2	52
A.12	Điều khiển IR cho điều hòa	53

A.13 Firmware gateway	54
A.14 Quản lý reconnect	54
A.15 Quy ước log	55
A.16 Bảng cấu hình đề xuất	55
B PHỤ LỤC KIỂM THỬ VÀ HƯỚNG DẪN VẬN HÀNH	56
B.1 Mục tiêu kiểm thử	56
B.2 Kiểm thử khối nguồn	56
B.3 Kiểm thử cảm biến	57
B.4 Kiểm thử LoRa	58
B.5 Kiểm thử ThingsBoard	58
B.6 Kiểm thử app Flutter	59
B.7 Kiểm thử actuator	59
B.8 Kịch bản vận hành trong căn hộ	60
B.9 Phân tích dữ liệu mẫu từ ảnh app	60
B.10 Bảng ngưỡng cảnh báo đề xuất	61
B.11 Hướng dẫn vận hành	61
B.12 Bảo trì hệ thống	62
B.13 Danh sách lỗi thường gặp	62
B.14 Tiêu chí nghiệm thu	63
B.15 Đề xuất cải tiến sau kiểm thử	63
B.16 Mẫu nhật ký kiểm thử	64
B.17 Phiếu hiệu chuẩn cảm biến pH	64
B.18 Phiếu hiệu chuẩn TDS	65
B.19 Bộ luật điều khiển tự động đề xuất	66
B.20 Checklist lắp đặt node trong căn hộ	66
B.21 Checklist bảo mật triển khai	67
B.22 Mẫu biên bản nghiệm thu	68

Danh sách hình vẽ

3.1	Kiến trúc phần cứng tổng thể của node	16
3.2	Luồng cấp nguồn chính của node	19
3.3	Chu kỳ hoạt động cơ bản của node	23
3.4	Nguyên lý điều khiển relay của node	28
4.1	Kiến trúc tổng thể của hệ thống	31
4.2	Luồng dữ liệu hai chiều của hệ thống	32
4.3	Minh họa màn hình đăng nhập của ứng dụng	39
4.4	Minh họa màn hình Sensor hiển thị dữ liệu node	40
4.5	Minh họa màn hình Actuator điều khiển relay và điều hòa	40

Bảng ký hiệu và chữ viết tắt

Ký hiệu

C_{CO_2} Nồng độ CO₂

E Cường độ ánh sáng

H Độ ẩm tương đối của không khí

Q Chỉ số chất lượng môi trường tổng hợp

T Nhiệt độ không khí trong phòng

V_{pH} Điện áp đầu ra của cảm biến pH

V_{TDS} Điện áp đầu ra của cảm biến TDS

Chữ viết tắt

ADC Analog-to-Digital Converter - Bộ chuyển đổi tương tự sang số

API Application Programming Interface - Giao diện lập trình ứng dụng

CO₂ Carbon dioxide - Khí cacbon dioxit

GPIO General Purpose Input/Output - Chân vào/ra đa dụng

HTTP HyperText Transfer Protocol - Giao thức truyền siêu văn bản

IAQ Indoor Air Quality - Chất lượng không khí trong nhà

IoT Internet of Things - Internet vạn vật

IR Infrared - Hồng ngoại

LoRa Long Range - Công nghệ truyền thông không dây tầm xa, công suất thấp

MQTT Message Queuing Telemetry Transport - Giao thức truyền bản tin nhẹ cho IoT

pH Potential of Hydrogen - Chỉ số axit/kiềm của dung dịch

PWM Pulse Width Modulation - Điều chế độ rộng xung

TDS Total Dissolved Solids - Tổng chất rắn hòa tan

GIỚI THIỆU

Chương này đóng vai trò thiết lập nền tảng lý luận cho toàn bộ khóa luận, tổng quan về bối cảnh nghiên cứu và các cơ sở khoa học cốt lõi dẫn dắt đến hướng tiếp cận của đề tài. Đề tài xoay quanh sự hội tụ đa ngành: kết hợp chặt chẽ giữa kỹ thuật giám sát vi khí hậu hiện đại, công nghệ thủy canh sinh thái (với trọng tâm là cây lan ý) và các hệ thống tự động hóa. Tất cả các thành tố này được ứng dụng trong một mạng lưới Internet vạn vật (IoT) bảo đảm tính đồng bộ, độ trễ thấp và khả năng vận hành tự chủ ở quy mô cục bộ.

1.1 Đặt vấn đề

Trong kỷ nguyên đô thị hóa hiện đại, chất lượng không khí trong không gian kín (IAQ) đã vượt ra khỏi khuôn khổ của những tiện nghi, trở thành một yếu tố có tác động trực tiếp và sâu sắc đến sức khỏe cộng đồng, năng lực nhận thức cũng như trạng thái cân bằng sinh học của con người. Các báo cáo uy tín liên tục cảnh báo rằng, sự suy giảm chất lượng không khí tại các không gian kín, kết hợp với chỉ số lưu thông khí tươi thấp, là nguyên nhân sâu xa kích hoạt và làm trầm trọng thêm các bệnh lý hô hấp mãn tính, hội chứng bệnh nhà kín (Sick Building Syndrome), và suy giảm chất lượng sống tổng thể [6, 7].

Nhìn vào thực tế kiến trúc của các căn hộ chung cư đương đại, chúng ta dễ dàng nhận thấy xu hướng tối đa hóa diện tích sử dụng bằng sự khép kín. Tuy nhiên, sự khép kín này lại tạo ra sự phụ thuộc vào các hệ thống điều hòa không khí, quạt thông gió cưỡng bức. Khi mật độ sinh hoạt của con người gia tăng trong các không gian hạn hẹp, tốc độ tích tụ của khí CO_2 diễn ra cực kỳ nhanh chóng. Kéo theo đó là sự xáo trộn liên tục của nhiệt độ và độ ẩm dưới tác động của thiết bị cơ điện. Điều đáng quan ngại là những biến thiên vi khí hậu này thường diễn tiến

một cách âm thầm, vượt qua ngưỡng dung sai và độ nhạy cảm của các giác quan thông thường, khiến con người mất đi khả năng nhận diện và can thiệp kịp thời.

Đứng trước thách thức đó, việc đưa các mảng xanh thực vật vào không gian sống đang thu hút sự quan tâm lớn của giới học thuật, không chỉ dưới lăng kính thẩm mỹ kiến trúc mà còn như một cỗ máy lọc khí sinh thái đầy hứa hẹn. Trong số các loài thực vật nội thất, lan ý (*Spathiphyllum*) đặc biệt nổi bật nhờ phổ thích ứng sinh lý cực rộng. Loài cây này có khả năng quang hợp hiệu quả ở cường độ sáng thấp, chịu đựng tốt các cú sốc nhiệt ẩm, và đặc biệt là sinh trưởng cực kỳ ổn định trong môi trường thủy canh [18, 17]. Các nghiên cứu chuyên sâu về sinh lý thực vật đã làm sáng tỏ khả năng của lan ý trong việc đồng hóa CO₂ và chủ động hấp thụ các hợp chất hữu cơ dễ bay hơi (VOC) độc hại—như benzene, formaldehyde, hay trichloroethylene—vốn rất phổ biến trong vật liệu nội thất [15, 8].

Dù vậy, khi xét dưới góc độ của hệ thống, năng lực xử lý sinh học của thực vật không phải là một hằng số. Nó là hàm phụ thuộc vào các biến số vi khí hậu như: động thái trao đổi chất tại vùng rễ, cường độ bức xạ quang hợp hiệu dụng và trạng thái hóa lý của dung dịch dinh dưỡng. Nhận thức sâu sắc được thực tiễn này, nghiên cứu của em không nhìn nhận lan ý như một bộ lọc khí thụ động để thay thế hoàn toàn các thiết bị cơ điện. Thay vào đó, em định vị hệ sinh thái thực vật này như một thành phần chủ động và được tinh chỉnh liên tục bởi một hệ thống kiểm soát môi trường tự động hóa.

Xuất phát từ những luận điểm trên, khóa luận đề xuất một hệ thống IoT đa mục tiêu: vừa theo dõi sát sao sự biến thiên của chất lượng không khí trong không gian sống, vừa giám sát chặt chẽ các thông số sinh lý của cây thủy canh. Về mặt kiến trúc vật lý, hệ thống là một mạng lưới các cảm biến thông minh sử dụng vi điều khiển Arduino Pro Mini, giao tiếp không dây qua giao thức tầm xa LoRa để truyền dữ liệu về gateway. Tại đây, luồng dữ liệu được định tuyến lên nền tảng đám mây ThingsBoard nhằm mục đích lưu trữ dài hạn, phân tích trực quan hóa và thiết lập cổng giao tiếp với ứng dụng di động Flutter dành cho người dùng cuối. Không dừng lại ở việc quan sát thụ động, giá trị cốt lõi của đề tài nằm ở việc tích hợp cơ chế phản hồi chủ động. Thông qua các rơ-le điều khiển quạt, đèn và module phát hồng ngoại điều khiển điều hòa nhiệt độ, em đã hiện thực hóa một vòng lặp điều khiển khép kín: đo lường chính xác, cảnh báo kịp thời, và tự động kích hoạt các kịch bản can thiệp thích nghi nhằm duy trì trạng thái vi khí hậu tối ưu nhất.

1.2 Tổng quan nghiên cứu

Định hướng tiếp cận truyền thống trong việc kiểm soát chất lượng không khí trong nhà (IAQ) phần lớn phụ thuộc vào các hệ thống thông gió cơ khí, thiết bị màng lọc tĩnh điện hoặc hệ thống điều hòa không khí. Mặc dù các giải pháp kỹ thuật này là nền tảng tất yếu, chúng thường đi kèm với chi phí năng lượng đáng kể, yêu cầu bảo trì liên tục và chưa giải quyết được nhu cầu thiết lập một hệ sinh thái không gian sống xanh trong bối cảnh kiến trúc hiện đại. Dưới lăng kính của công nghệ sinh thái, các phân tích chuyên sâu về hệ thống thủy canh tích hợp Internet vạn vật (IoT) đã minh chứng rằng, thực vật có thể được số hóa để trở thành một cấu phần chủ động trong mạng lưới giám sát vi khí hậu, nơi các biến số như nồng độ CO₂, nhiệt - ẩm độ, quang năng và đặc tính dinh dưỡng được lượng hóa theo thời gian thực [15]. Xét riêng về mặt thực vật học ứng dụng, lan ý (*Spathiphyllum*) được định vị là loài sinh vật chỉ thị thích hợp nhờ biên độ chịu đựng tốt với ánh sáng khuếch tán, yêu cầu môi trường ẩm ẩm và khả năng sinh trưởng bền vững trong môi trường thủy canh [18, 17].

Trong kỷ nguyên của nhà thông minh (smart home), các hệ thống tự động hóa hiện hành chủ yếu vận hành xoay quanh trục điều khiển thiết bị chấp hành (chiếu sáng, thông gió, an ninh). Một bộ phận các hệ thống tiên tiến có tích hợp module cảm biến nhiệt - ẩm và CO₂; tuy nhiên, luồng dữ liệu môi trường này phần lớn được xử lý như một tham số độc lập, thiếu đi sự tham chiếu và liên kết với bài toán sinh thái học của thực vật. Ở chiều ngược lại, các hệ thống nông nghiệp chính xác (precision agriculture) lại chỉ tập trung vào sinh lý cây trồng (độ ẩm giá thể, pH, độ dẫn điện TDS, quang chu kỳ) mà bỏ qua sự tương tác mật thiết của chúng với chất lượng không khí xung quanh. Sự phân mảnh trong tư duy thiết kế này đã làm suy giảm tiềm năng của mô hình thực vật hỗ trợ IAQ, bởi lẽ động lực học vùng rễ, trạng thái cân bằng dinh dưỡng và sức khỏe tổng thể của cây có mối tương quan tuyến tính với khả năng trao đổi khí và cải thiện vi khí hậu của chúng.

Trên bình diện công nghệ, kiến trúc IoT cung cấp bộ khung hạ tầng cho phép hội tụ các module cảm biến, thiết bị chấp hành, vi điều khiển cục bộ và nền tảng điện toán đám mây thành một cơ chế giám sát thời gian thực liền mạch [9, 11, 12]. Khi được tinh chỉnh cho mô hình lan ý thủy canh, hạ tầng IoT có năng lực lượng hóa sự sai lệch của các tham số môi trường và dinh dưỡng (ánh sáng, nồng độ ion, pH, nhiệt - ẩm độ), qua đó thiết lập các tập luật cảnh báo và kích hoạt chu trình điều khiển vòng kín. Theo các nghiên cứu thực nghiệm sinh lý học

trên *Spathiphyllum*, điều kiện tối ưu trong giai đoạn thích nghi bao gồm độ dẫn điện (EC) dung dịch ở mức 1.2 dS/m, pH xấp xỉ 5.8, mật độ quang thông (PPF) trong khoảng 70–100 $\mu\text{mol m}^{-2} \text{s}^{-1}$ và giới hạn nhiệt độ quanh mức 25 °C [16]. Song song đó, các cảm nang hướng dẫn ứng dụng thủy canh vi mô khuyến nghị duy trì dải pH từ 6.0–6.5 và kiểm soát nồng độ TDS định kỳ [17]. Mặc dù lý thuyết đã được khẳng định, các nguyên mẫu hệ thống hiện hành vẫn chưa giải quyết triệt để bài toán đồng bộ hóa luồng điều khiển hai chiều giữa giao diện người dùng cuối, lõi xử lý IoT và cụm thiết bị viền (edge devices).

Xét về hạ tầng truyền thông, giao thức Wi-Fi thường là ưu tiên mặc định trong không gian dân dụng nhờ tính sẵn sàng cao. Dẫu vậy, việc bố trí các node cảm biến tại các khu vực rìa (ban công, cửa sổ) hoặc tại các không gian bị phân tán bởi kết cấu vật lý thường dẫn đến suy hao tín hiệu và mất ổn định liên kết. Ngược lại, kỹ thuật điều chế LoRa thể hiện tính ưu việt vượt trội đối với việc truyền tải các gói dữ liệu telemetry chu kỳ, kích thước nhỏ và không đòi hỏi băng thông rộng. Do đó, kiến trúc mạng của đề tài tận dụng liên kết LoRa ở tầng thấp (node - gateway), trong khi giao phó nhiệm vụ định tuyến dữ liệu lên nền tảng đám mây ThingsBoard cho gateway ESP32.

Trong thiết kế phần mềm, ThingsBoard được khai thác như một nền tảng lõi đảm nhiệm chức năng quản lý danh tính thiết bị, lưu trữ chuỗi thời gian telemetry, cung cấp công cụ phân tích trực quan và kiến tạo cơ chế RPC (Remote Procedure Call) cũng như attribute cho luồng điều khiển song hướng. Ở phía người dùng cuối, một ứng dụng di động phát triển trên nền tảng Flutter sẽ giao tiếp trực tiếp qua các điểm cuối API của ThingsBoard để kết xuất dữ liệu, cấu hình ngưỡng hoạt động và khởi tạo lệnh điều khiển. Cấu trúc thiết kế này đảm bảo nguyên tắc phân rã trách nhiệm (separation of concerns): bóc tách rõ ràng giữa lớp biên (thu thập dữ liệu), lớp mạng (truyền thông không dây), lớp lõi (xử lý dữ liệu đám mây) và lớp ứng dụng (giao diện tương tác).

Dựa trên những cơ sở luận lý và thực tiễn vừa trình bày, nghiên cứu này đặt trọng tâm vào việc thiết kế và hiện thực hóa một nguyên mẫu IoT tích hợp. Nguyên mẫu này không chỉ đảm bảo tính khả thi về chi phí và quy mô triển khai trong căn hộ chung cư, mà còn hiện thực hóa sự đồng bộ giữa hệ thống quản trị vi khí hậu IAQ và mô hình chăm sóc thực vật tự động hóa. Đề tài không dừng lại ở mức độ thu thập và biểu diễn số liệu, mà còn cung cấp khả năng cảnh báo sớm, can thiệp thủ công từ xa và thiết lập nền tảng tham số vững chắc để hướng

tối việc triển khai các giải thuật điều khiển thích nghi trong các pha phát triển kế tiếp.

1.3 Mục tiêu của đề tài

Mục tiêu cốt lõi của khóa luận là thiết kế, chế tạo và đánh giá một hệ thống Internet vạn vật (IoT) đa nhiệm, tích hợp khả năng đo lường động học vi khí hậu trong không gian căn hộ, quản trị điều kiện sinh trưởng của thực vật thủy canh (lan ý), đồng thời thiết lập cơ chế điều khiển chủ động đối với các thiết bị viên nhằm tối ưu hóa môi trường sống. Cần minh định rằng, giá trị lõi của nghiên cứu không nằm ở nỗ lực chứng minh hệ thực vật có khả năng thay thế hoàn toàn các giải pháp cơ điện lọc khí chuyên dụng. Thay vào đó, đề tài tập trung lượng giá tính khả thi và độ tin cậy của một kiến trúc tích hợp đa phương diện: bao gồm việc số hóa các chỉ số IAQ, quản lý tự động hệ sinh thái thủy canh vi mô và thiết lập vòng lặp điều khiển phản hồi theo thời gian thực.

Để hiện thực hóa định hướng này, các mục tiêu cụ thể được xác định như sau:

- Thiết kế và chế tạo các cụm node cảm biến ngoại vi dựa trên nền tảng vi xử lý Arduino Pro Mini, đảm nhiệm việc trích xuất liên tục các tham số hóa sinh và vi khí hậu bao gồm: độ dẫn điện TDS, nồng độ ion pH, nhiệt độ môi trường, độ ẩm tương đối, phân áp nồng độ CO₂ và cường độ bức xạ quang học.
- Thiết lập giao thức truyền thông tầm xa, công suất thấp (LoRa) nhằm bảo đảm tính toàn vẹn và độ tin cậy của kênh truyền dữ liệu từ các node biên đến gateway trung tâm, vượt qua các giới hạn suy hao của hạ tầng mạng cục bộ.
- Xây dựng hệ thống gateway ứng dụng vi điều khiển ESP32 với chức năng giải mã gói tin LoRa, xác thực tính hợp lệ của dữ liệu và định tuyến chuỗi thời gian telemetry lên kiến trúc đám mây ThingsBoard.
- Phát triển hệ sinh thái phần mềm giao diện người dùng trên nền tảng di động Flutter, tích hợp với API của ThingsBoard nhằm cung cấp khả năng trực quan hóa dữ liệu biểu đồ, giám sát trạng thái cảnh báo, truy xuất lược sử thông số và linh hoạt cấu hình các ngưỡng vận hành biên.
- Tích hợp khả năng điều khiển thiết bị chấp hành ngay tại node biên, cho phép thao tác đóng/cắt module relay (điều khiển quạt thông gió, hệ thống chiếu

sáng) và điều chế mã hồng ngoại (điều khiển bù trừ nhiệt độ thông qua hệ thống điều hòa).

- Xây dựng và đề xuất tập luật điều khiển logic dựa trên biên độ dao động của các tham số đầu vào (nồng độ CO₂, mức độ chiếu sáng, gradient nhiệt - ẩm, cũng như các chỉ số dung dịch TDS và pH), qua đó song hành cải thiện các chỉ tiêu IAQ và duy trì môi trường sống lý tưởng cho hệ thực vật.
- Khảo nghiệm, hiệu chỉnh và đánh giá năng lực vận hành của nguyên mẫu hệ thống thông qua các kịch bản thực chứng trong môi trường căn hộ chung cư tiêu chuẩn.

1.4 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của khóa luận là hệ thống giám sát và điều khiển môi trường trong nhà dựa trên cảm biến, truyền thông không dây, nền tảng IoT và ứng dụng di động. Hệ thống được định hướng cho căn hộ chung cư, nơi số lượng node không lớn nhưng yêu cầu tính ổn định, khả năng lắp đặt thuận tiện và giao diện sử dụng rõ ràng. Trong hệ thống này, cây lan ý được xem là thành phần sinh học hỗ trợ cải thiện vi khí hậu trong điều kiện chăm sóc phù hợp; phần IoT giữ vai trò đo lường, lưu trữ dữ liệu, cảnh báo và điều khiển.

Phạm vi nghiên cứu tập trung vào một node cảm biến và một gateway. Node được bố trí gần khu vực trồng lan ý hoặc khu vực sinh hoạt cần giám sát; các thông số đo bao gồm TDS, pH, nhiệt độ, độ ẩm, CO₂ và ánh sáng. Phần điều khiển gồm quạt, đèn thông qua relay và điều hòa thông qua phát lệnh hồng ngoại. Khóa luận không đi sâu vào mô hình sinh học của cơ chế hấp thụ VOC, không định lượng tốc độ hấp thụ từng chất ô nhiễm và không đặt mục tiêu thay thế thiết bị đo IAQ chuyên dụng. Thay vào đó, đề tài tập trung xây dựng một nguyên mẫu có khả năng thu thập dữ liệu, đưa ra cảnh báo và thực hiện điều khiển trong điều kiện căn hộ thực tế.

1.5 Phương pháp nghiên cứu

Khóa luận sử dụng phương pháp nghiên cứu kết hợp giữa khảo sát tài liệu, thiết kế hệ thống và triển khai thực nghiệm. Trước hết, các thông số liên quan đến

chất lượng không khí trong nhà và điều kiện sinh trưởng của cây được tổng hợp từ các tài liệu về IAQ, IoT và hệ thống trồng cây/thủy canh thông minh [6, 7, 10]. Trên cơ sở đó, đề tài xác định nhóm cảm biến cần triển khai gồm CO₂, nhiệt độ, độ ẩm, ánh sáng, TDS và pH. Kiến trúc phần cứng và phần mềm sau đó được thiết kế theo hướng phân lớp, trong đó node đảm nhiệm thu thập dữ liệu, gateway thực hiện truyền thông lên nền tảng IoT, ThingsBoard quản lý dữ liệu và ứng dụng Flutter phục vụ người dùng cuối.

Sau khi lựa chọn phần cứng, khóa luận xây dựng định dạng gói tin LoRa, quy trình gửi nhận dữ liệu, cơ chế xác nhận lệnh và mô hình dữ liệu trên ThingsBoard. Hệ thống được kiểm thử theo từng khối chức năng trước khi kiểm thử tích hợp toàn bộ luồng từ cảm biến đến ứng dụng và từ ứng dụng quay lại thiết bị chấp hành. Kết quả thử nghiệm được phân tích theo ba nhóm tiêu chí: khả năng thu thập và hiển thị dữ liệu theo thời gian thực, khả năng truyền lệnh điều khiển hai chiều, và các hạn chế liên quan đến hiệu chuẩn cảm biến, độ ổn định truyền thông cũng như khả năng mở rộng.

1.6 Ý nghĩa thực tiễn

Hệ thống đề xuất có ý nghĩa thực tiễn ở khả năng giúp người dùng nhận biết rõ hơn trạng thái môi trường sống trong căn hộ. Thông qua các thông số được cập nhật theo thời gian thực, người dùng có thể phát hiện sớm các tình huống như CO₂ tăng cao, ánh sáng không phù hợp hoặc điều kiện chăm sóc cây chưa đạt yêu cầu. Việc tích hợp điều khiển quạt, đèn và điều hòa giúp hệ thống không chỉ dừng ở mức ghi nhận dữ liệu mà còn hỗ trợ phản ứng kịp thời trước các thay đổi của môi trường.

Về mặt kỹ thuật, đề tài minh họa một mô hình IoT tương đối hoàn chỉnh, bắt đầu từ node cảm biến tài nguyên hạn chế, qua truyền thông LoRa và gateway ESP32, đến nền tảng ThingsBoard và ứng dụng Flutter. Kiến trúc này có thể được mở rộng cho nhiều node hoặc điều chỉnh để áp dụng cho các bài toán giám sát môi trường trong văn phòng, phòng học, phòng ngủ và khu vực trồng cây quy mô nhỏ.

1.7 Bố cục khóa luận

Khóa luận gồm năm phần chính. Chương 1 giới thiệu bối cảnh, mục tiêu, phạm vi và phương pháp nghiên cứu. Chương 2 trình bày cơ sở lý thuyết về chất lượng không khí trong nhà, cây trồng trong nhà, cảm biến, LoRa, ESP32, ThingsBoard và Flutter. Chương 3 trình bày thiết kế và triển khai hệ thống gồm node cảm biến, gateway, nền tảng IoT và ứng dụng di động. Chương 4 trình bày kết quả kiểm thử, đánh giá và thảo luận. Phần cuối là kết luận và hướng phát triển tiếp theo.

CƠ SỞ LÝ THUYẾT

Chương này trình bày các cơ sở lý thuyết và công nghệ nền tảng phục vụ quá trình thiết kế hệ thống. Nội dung được triển khai theo hai nhóm chính: cơ sở về chất lượng không khí trong nhà và điều kiện chăm sóc cây lan ý; cơ sở công nghệ gồm cảm biến, truyền thông LoRa, vi điều khiển Arduino Pro Mini, gateway ESP32, nền tảng ThingsBoard và ứng dụng Flutter.

2.1 Chất lượng không khí trong nhà

Chất lượng không khí trong nhà (Indoor Air Quality - IAQ) phản ánh mức độ phù hợp của môi trường không khí đối với sức khỏe, sự thoải mái và hiệu quả sinh hoạt của con người. Trong căn hộ chung cư, IAQ chịu tác động đồng thời của thông gió, số người trong phòng, hoạt động nấu nướng, thiết bị điện, vật liệu nội thất, nhiệt độ, độ ẩm và thói quen sử dụng điều hòa. Do các yếu tố này thay đổi theo thời gian, việc giám sát liên tục có ý nghĩa hơn so với đánh giá cảm quan tại một thời điểm.

Trong phạm vi khóa luận, nhóm thông số môi trường được giám sát gồm nhiệt độ, độ ẩm, nồng độ CO₂ và ánh sáng. Nhiệt độ và độ ẩm vừa ảnh hưởng đến cảm giác tiện nghi của người sử dụng, vừa liên quan đến khả năng sinh trưởng của lan ý. CO₂ được sử dụng như chỉ báo gián tiếp cho mức độ thông gió và mật độ người trong phòng; khi CO₂ tăng cao, hệ thống có thể phát cảnh báo hoặc đề xuất bật quạt thông gió. Ánh sáng được xem xét ở cả hai khía cạnh: điều kiện sinh hoạt của con người và điều kiện quang hợp của cây trồng trong nhà.

2.1.1 Nhiệt độ và độ ẩm

Nhiệt độ quá cao hoặc quá thấp đều có thể gây khó chịu và làm tăng nhu cầu sử dụng điều hòa. Độ ẩm thấp thường gây khô da, khô đường hô hấp, trong khi độ ẩm quá cao tạo điều kiện cho nấm mốc phát triển. Đối với lan ý, tài liệu chăm sóc khuyến nghị môi trường ẩm, khoảng 65–80 °F, độ ẩm từ khoảng 50% trở lên và tránh các luồng gió nóng hoặc lạnh thổi trực tiếp vào cây [18]. Do đó, nhiệt độ và độ ẩm không chỉ là thông số tiện nghi mà còn là cơ sở để đánh giá điều kiện chăm sóc cây trong hệ thống.

2.1.2 Nồng độ CO₂

Trong không gian căn hộ, CO₂ chủ yếu phát sinh từ hoạt động hô hấp của con người. Khi phòng kín hoặc thông gió kém, nồng độ CO₂ có xu hướng tăng và phản ánh mức độ trao đổi không khí với bên ngoài. Các tài liệu về IAQ xem CO₂ là một chỉ báo quan trọng đối với ô nhiễm có nguồn gốc từ người sử dụng; từ khoảng 1000 ppm, cảm giác khó chịu, mệt mỏi hoặc giảm tập trung có thể xuất hiện ở một bộ phận người trong phòng [19]. Vì vậy, hệ thống sử dụng CO₂ như một ngưỡng cảnh báo để đề xuất mở cửa, bật quạt thông gió hoặc điều chỉnh cách sử dụng phòng.

2.1.3 Hợp chất bay hơi và giới hạn của hệ thống

Ngoài CO₂, IAQ còn chịu ảnh hưởng bởi bụi mịn và các hợp chất hữu cơ bay hơi (VOC) phát sinh từ vật liệu nội thất, sơn, chất tẩy rửa hoặc hoạt động nấu nướng [7]. Một số nghiên cứu về cây trồng trong nhà cho thấy cây có thể hỗ trợ hấp thụ một phần chất ô nhiễm trong điều kiện thí nghiệm [8]. Đối với lan ý, tài liệu về hệ thủy canh cho IAQ đề cập *Spathiphyllum* như loài cây có tiềm năng hỗ trợ hấp thụ CO₂ và VOC khi được duy trì trong điều kiện sinh trưởng phù hợp [15]. Tuy nhiên, nguyên mẫu của khóa luận chưa đo trực tiếp VOC hoặc bụi mịn; vì vậy, hệ thống tập trung vào CO₂, nhiệt độ, độ ẩm, ánh sáng và các thông số chăm sóc cây như nhóm chỉ báo ban đầu cho môi trường căn hộ.

2.2 Cây lan ý và các thông số chăm sóc

Cây lan ý (*Spathiphyllum*), còn được biết đến với tên tiếng Anh là peace lily, là loài cây nội thất phổ biến nhờ hình thái lá xanh bóng, hoa trắng dạng mo và khả năng thích nghi tương đối tốt trong nhà. Tài liệu chăm sóc cây cho thấy lan ý có thể sống trong điều kiện ánh sáng yếu, song cây sinh trưởng và ra hoa tốt hơn khi nhận ánh sáng gián tiếp sáng; cây cũng ưa môi trường ẩm, ẩm và cần tránh ánh nắng trực tiếp mạnh [18]. Những đặc điểm này phù hợp với căn hộ chung cư, nơi cây thường được đặt ở gần cửa sổ, ban công có che chắn hoặc khu vực sinh hoạt có ánh sáng khuếch tán.

Về vai trò đối với IAQ, lan ý cần được nhìn nhận thận trọng. Cây có thể là thành phần hỗ trợ trong hệ kiểm soát môi trường, nhưng không thay thế thông gió, máy lọc không khí hoặc các thiết bị xử lý ô nhiễm chuyên dụng. Tài liệu về hệ thủy canh cho IAQ xếp lan ý vào nhóm cây có tiềm năng hỗ trợ hấp thụ CO₂ và VOC; khi kết hợp với IoT, các thông số như CO₂, nhiệt độ, độ ẩm, ánh sáng và dinh dưỡng có thể được theo dõi nhằm duy trì điều kiện sinh trưởng ổn định cho cây [15]. Vì vậy, khóa luận sử dụng IoT để giám sát lan ý như một phần của môi trường sống, đồng thời liên hệ dữ liệu chăm sóc cây với dữ liệu IAQ xung quanh.

2.2.1 TDS và pH

TDS biểu thị tổng lượng chất rắn hòa tan trong nước hoặc dung dịch dinh dưỡng. Trong thủy canh, EC thường được sử dụng như đại lượng quản lý dinh dưỡng vì các muối hòa tan tạo thành ion và làm thay đổi khả năng dẫn điện của dung dịch. EC quá cao có thể gây stress thẩm thấu, mất cân bằng dinh dưỡng hoặc độc ion; ngược lại, EC quá thấp thường phản ánh tình trạng dung dịch nghèo dinh dưỡng [14]. Trong nguyên mẫu của khóa luận, cảm biến TDS được dùng như một chỉ báo thực dụng để phát hiện dung dịch quá loãng hoặc quá đậm, từ đó gợi ý người dùng thay nước, bổ sung nước hoặc kiểm tra lại dung dịch dinh dưỡng.

pH phản ánh tính axit hoặc kiềm của dung dịch. Đối với lan ý thủy canh quy mô hộ gia đình, tài liệu hướng dẫn khuyến nghị duy trì dung dịch hơi axit, khoảng pH 6.0–6.5 [17]; trong nghiên cứu thủy canh trên *Spathiphyllum*, dung dịch được duy trì ở pH 5.8 và EC 1.2 dS/m trong các thí nghiệm sinh trưởng [16]. Khi pH lệch khỏi vùng phù hợp, cây vẫn có nước và ánh sáng nhưng khả năng hấp thụ một số chất dinh dưỡng có thể suy giảm. Vì vậy, đo pH giúp hệ thống đánh giá

trạng thái dung dịch và đưa ra cảnh báo chăm sóc cây kịp thời hơn.

2.2.2 Ánh sáng

Ánh sáng quyết định trực tiếp đến quang hợp và gián tiếp ảnh hưởng đến hình thái sinh trưởng của cây. Lan ý có thể tồn tại trong điều kiện ánh sáng thấp, nhưng ánh sáng gián tiếp sáng thường giúp cây sinh trưởng và ra hoa tốt hơn; ngược lại, nắng trực tiếp mạnh có thể làm hư hại lá [18]. Trong nghiên cứu thủy canh trên *Spathiphyllum*, mức PPF 70–100 $\mu\text{mol m}^{-2} \text{s}^{-1}$ cho đáp ứng sinh trưởng và quang hợp tốt hơn so với các mức quá thấp hoặc quá cao [16]. Do vị trí đặt cây trong căn hộ có thể bị che khuất hoặc xa nguồn sáng tự nhiên, cảm biến ánh sáng được sử dụng để đánh giá điều kiện chiếu sáng và làm cơ sở điều khiển đèn bổ sung.

2.2.3 Liên hệ giữa chăm sóc cây và IAQ

Trong hệ thống đề xuất, dữ liệu chăm sóc cây và dữ liệu IAQ không được xử lý như hai nhóm thông tin tách rời. Ánh sáng thấp kéo dài có thể làm giảm quang hợp; nhiệt độ hoặc độ ẩm không phù hợp vừa ảnh hưởng đến cây, vừa ảnh hưởng đến cảm giác tiện nghi của người sử dụng. Khi CO₂ tăng cao, hệ thống ưu tiên cảnh báo thông gió hoặc bật quạt; khi ánh sáng không đủ, hệ thống có thể đề xuất hoặc kích hoạt đèn bổ sung. Nhờ đó, các ngưỡng điều khiển được xây dựng để phục vụ đồng thời sức khỏe cây và chất lượng môi trường sống, thay vì chỉ tập trung vào nước và dinh dưỡng như nhiều hệ thống trồng cây thông minh đơn lẻ.

2.3 Node cảm biến Arduino Pro Mini

Arduino Pro Mini là bo vi điều khiển nhỏ gọn, tiêu thụ năng lượng thấp và phù hợp với các ứng dụng cảm biến. Trong hệ thống này, Arduino Pro Mini đảm nhiệm việc đọc giá trị từ các cảm biến TDS, pH, nhiệt độ, độ ẩm, CO₂ và ánh sáng, sau đó đóng gói dữ liệu để gửi qua module LoRa.

Các cảm biến analog như TDS, pH và ánh sáng được đọc thông qua bộ chuyển đổi ADC của vi điều khiển. Các cảm biến digital như nhiệt độ, độ ẩm hoặc CO₂ có thể giao tiếp qua GPIO, UART hoặc giao thức chuyên dụng tùy loại module

sử dụng. Node cũng điều khiển relay và bộ phát hồng ngoại IR khi nhận lệnh từ gateway.

2.3.1 Yêu cầu đối với node

Node cần đáp ứng các yêu cầu: đọc dữ liệu ổn định, đóng gói dữ liệu ngắn gọn, truyền LoRa định kỳ, nhận lệnh điều khiển, phản hồi trạng thái và có khả năng hoạt động lâu dài trong môi trường căn hộ. Do Arduino Pro Mini có tài nguyên hạn chế, chương trình trên node cần đơn giản, tránh xử lý phức tạp và ưu tiên độ tin cậy.

2.4 Truyền thông LoRa và gateway ESP32

LoRa là công nghệ truyền thông không dây công suất thấp, phù hợp với các gói dữ liệu nhỏ và khoảng cách truyền xa. Trong căn hộ, LoRa giúp node cảm biến đặt tại vị trí xa router Wi-Fi vẫn có thể gửi dữ liệu ổn định về gateway. Hai module LoRa được sử dụng: một module gắn với node Arduino Pro Mini và một module gắn với gateway ESP32.

ESP32 có ưu điểm tích hợp Wi-Fi, Bluetooth, hiệu năng xử lý tốt và nhiều chân giao tiếp ngoại vi. Trong hệ thống, ESP32 đóng vai trò gateway: nhận gói tin từ LoRa, kiểm tra cấu trúc dữ liệu, chuyển đổi thành telemetry và gửi lên ThingsBoard thông qua kết nối Internet. Gateway cũng nhận lệnh điều khiển từ ThingsBoard và chuyển tiếp lệnh về node qua LoRa.

2.4.1 Luồng truyền dữ liệu

Luồng uplink bắt đầu từ cảm biến tại node, đi qua Arduino Pro Mini, module LoRa, gateway ESP32 và kết thúc tại ThingsBoard. Luồng downlink bắt đầu từ ứng dụng Flutter hoặc dashboard ThingsBoard, đi qua ThingsBoard, gateway ESP32, LoRa và đến node để điều khiển relay hoặc IR.

2.5 ThingsBoard và ứng dụng Flutter

ThingsBoard là nền tảng IoT hỗ trợ quản lý thiết bị, thu thập telemetry, hiển thị dashboard, cấu hình rule chain và điều khiển thiết bị từ xa. Trong khóa luận, gateway ESP32 gửi dữ liệu cảm biến lên ThingsBoard theo định dạng khóa - giá trị. Mỗi thông số như nhiệt độ, độ ẩm, CO2, ánh sáng, TDS và pH được lưu thành một telemetry riêng để dễ hiển thị và phân tích.

Flutter là framework phát triển ứng dụng di động đa nền tảng. Ứng dụng Flutter kết nối với ThingsBoard để lấy dữ liệu mới nhất, hiển thị biểu đồ, trạng thái cảnh báo và gửi lệnh điều khiển. Các lệnh chính gồm bật/tắt quạt, bật/tắt đèn, tăng nhiệt độ điều hòa và giảm nhiệt độ điều hòa. Khi người dùng thao tác trên app, lệnh được gửi đến ThingsBoard, gateway nhận lệnh và chuyển tiếp đến node.

THIẾT KẾ PHẦN CỨNG NODE

Chương này trình bày thiết kế phần cứng của node cảm biến và điều khiển. Nội dung được xây dựng dựa trên sơ đồ nguyên lý của mạch node, trong đó các khối chức năng chính gồm khối nguồn, khối vi điều khiển, khối giao tiếp, khối cảm biến và khối actuator. Việc tách hệ thống thành các khối giúp quá trình thiết kế, kiểm thử và sửa lỗi trở nên rõ ràng hơn, đồng thời làm cơ sở để mở rộng node trong các phiên bản sau.

3.1 Yêu cầu thiết kế phần cứng

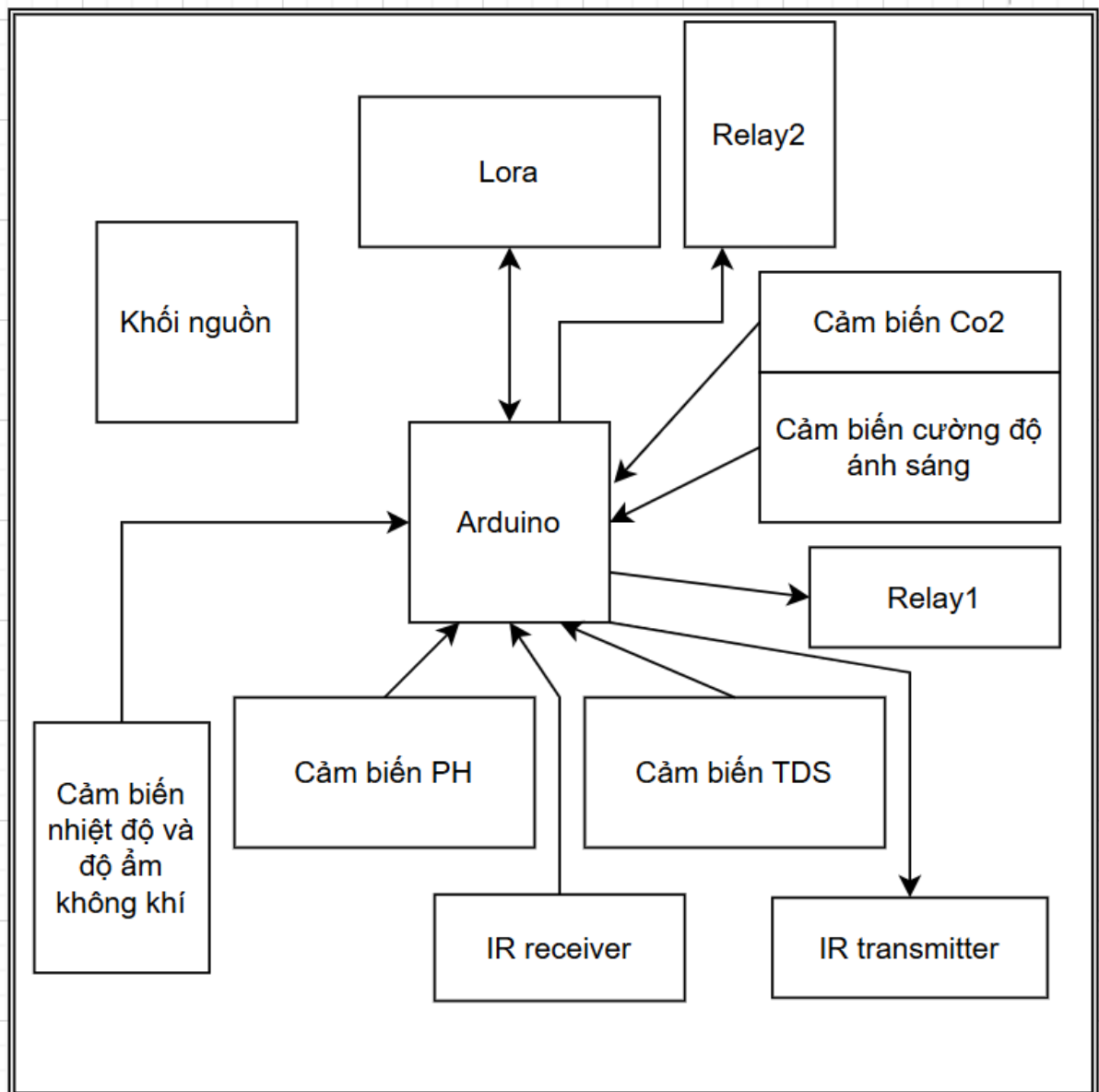
Node được thiết kế để hoạt động trong môi trường căn hộ, đặt gần cây trồng hoặc khu vực cần giám sát chất lượng không khí. Do đó phần cứng cần đáp ứng đồng thời các yêu cầu về đo lường, truyền thông và điều khiển thiết bị. Node phải đọc được dữ liệu từ nhiều cảm biến khác nhau, truyền dữ liệu về gateway qua LoRa, nhận lệnh điều khiển từ gateway và điều khiển các cơ cấu chấp hành như relay hoặc bộ phát hồng ngoại.

Các yêu cầu chính của phần cứng gồm:

- Cung cấp được các mức nguồn cần thiết cho vi điều khiển, cảm biến, LoRa và relay.
- Có khả năng đọc cảm biến analog như TDS, pH và ánh sáng.
- Có khả năng giao tiếp với cảm biến digital thông qua I2C, UART hoặc GPIO.
- Có kênh giao tiếp LoRa để truyền dữ liệu không dây về gateway ESP32.
- Có mạch relay để đóng cắt quạt, đèn hoặc thiết bị ngoài.
- Có mạch phát hồng ngoại để điều khiển điều hòa.

- Có đầu nối mở rộng để thuận tiện đấu dây cảm biến và kiểm thử.

Vì node sử dụng Arduino Pro Mini nên tài nguyên xử lý, số chân vào ra và bộ nhớ đều có giới hạn. Điều này ảnh hưởng trực tiếp đến cách bố trí chân, định dạng gói tin và thuật toán xử lý trên node. Thiết kế phần cứng cần ưu tiên sự ổn định và dễ kiểm tra hơn là tích hợp quá nhiều chức năng phức tạp vào một node duy nhất.



Hình 3.1: Kiến trúc phần cứng tổng thể của node

3.2 Phân rã chức năng theo khối

Từ sơ đồ nguyên lý, node được chia thành năm khối chính. Khối nguồn nhận điện áp đầu vào và tạo ra các mức nguồn ổn định. Khối vi điều khiển sử dụng Arduino Pro Mini làm trung tâm xử lý. Khối giao tiếp gồm LoRa RA-02 và mạch giao tiếp RS485 dự phòng hoặc mở rộng. Khối cảm biến gồm các đầu nối cho cảm biến TDS, pH, ánh sáng, nhiệt độ, độ ẩm và CO2. Khối actuator gồm relay và mạch phát IR.

Khối	Thành phần chính	Vai trò trong hệ thống
Khối nguồn	Jack DC, diode bảo vệ, module hạ áp, AMS1117-3.3V, tụ lọc, LED báo nguồn	Tạo các mức nguồn 24V, 5V và 3.3V; bảo vệ mạch khỏi đấu ngược cực và giảm nhiễu nguồn.
Khối vi điều khiển	Arduino Pro Mini, các header P1, P2, P4	Đọc cảm biến, xử lý dữ liệu, điều khiển relay/IR và giao tiếp với LoRa.
Khối giao tiếp	RA-02 LoRa, MAX485, header RS485	Truyền dữ liệu không dây về gateway và hỗ trợ giao tiếp mở rộng có dây khi cần.
Khối cảm biến	Header TDS, pH, ánh sáng, I2C, analog	Kết nối các cảm biến đo môi trường và điều kiện chăm sóc cây.
Khối actuator	Relay RL1, RL2, RL3; transistor; diode dập xung; LED IR	Bật/tắt quạt, đèn, tải ngoài và phát lệnh hồng ngoại điều khiển điều hòa.

Việc phân rã này cũng hỗ trợ quá trình kiểm thử. Khi một chức năng không hoạt động, có thể cô lập lỗi theo từng khối: kiểm tra nguồn trước, sau đó kiểm tra vi điều khiển, đường giao tiếp, cảm biến và cuối cùng là tải điều khiển. Đây là nguyên tắc quan trọng đối với các mạch IoT tự thiết kế vì lỗi có thể xuất phát từ cả phần cứng, dây nối, nhiễu nguồn lẫn chương trình điều khiển.

3.3 Nguyên tắc bố trí chân và tín hiệu

Arduino Pro Mini có số chân hạn chế, trong khi hệ thống cần kết nối nhiều ngoại vi. Do đó các chân được phân nhóm theo chức năng. Nhóm SPI dùng cho LoRa gồm MOSI, MISO, SCK, NSS, DI00 và RST. Nhóm analog dùng cho cảm biến TDS, pH, ánh sáng hoặc cảm biến analog khác. Nhóm digital dùng cho relay, IR LED, tín hiệu điều khiển MAX485 và các cảm biến digital.

Khi thiết kế, các tín hiệu có tốc độ cao hoặc nhạy nhiều cần được ưu tiên đường đi ngắn và rõ ràng. Tín hiệu SPI đến LoRa nên tránh chạy gần đường cấp relay hoặc tải công suất. Các đường analog từ cảm biến pH và TDS cần hạn chế nhiễu vì tín hiệu đo thường nhỏ và dễ bị ảnh hưởng bởi nguồn, relay hoặc dây dài.

Nhóm tín hiệu	Tín hiệu đại diện	Ghi chú thiết kế
SPI LoRa	MOSI, MISO, SCK, NSS	Dùng cho module RA-02, yêu cầu mức logic phù hợp với 3.3V.
Điều khiển LoRa	RST, DI00	Dùng reset module và nhận ngắt khi có gói tin.
Analog cảm biến	Po, A, OUT	Đọc pH, ánh sáng, TDS hoặc cảm biến analog tương tự.
Actuator	RL1, RL2, RL3, TX	Điều khiển relay và LED hồng ngoại.
I2C	SCL, SDA	Kết nối cảm biến nhiệt độ, độ ẩm hoặc module mở rộng.
UART/RS485	UITXo, UIRXo, UIENo	Kết nối MAX485 hoặc cảm biến công nghiệp nếu mở rộng.

Trong thiết kế thực tế, việc đặt tên net rõ ràng giúp giảm nhầm lẫn khi lập trình. Các tên như RL1, RL2, Po, TDS, DI00, NSS tạo liên hệ trực tiếp giữa sơ đồ nguyên lý và mã nguồn firmware. Đây là điểm quan trọng vì phần mềm và phần cứng của hệ thống được phát triển song song.

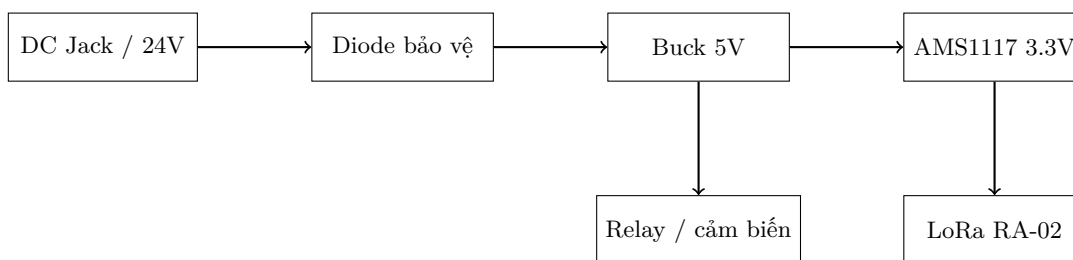
3.4 Thiết kế khối nguồn

Khối nguồn là nền tảng cho toàn bộ node. Theo sơ đồ nguyên lý, mạch có đầu vào nguồn ngoài qua jack DC và header cấp nguồn. Từ nguồn đầu vào, mạch tạo ra các rail điện áp phục vụ từng nhóm linh kiện: điện áp cao hơn cho tải hoặc relay, 5V cho một số cảm biến và mạch điều khiển, 3.3V cho module LoRa và các linh kiện logic mức thấp.

3.4.1 Nguồn đầu vào và bảo vệ

Đầu vào nguồn được thiết kế với jack DC và diode bảo vệ. Diode giúp hạn chế rủi ro khi người dùng đấu nhầm cực hoặc có xung ngược từ tải. Trong môi trường thử nghiệm, việc cắm nhầm adapter hoặc đảo cực dây cấp nguồn là lỗi phổ biến, vì vậy phần bảo vệ nguồn là cần thiết. Ngoài diode bảo vệ, mạch còn bố trí tụ lọc ở các rail nguồn để giảm dao động điện áp khi relay đóng cắt hoặc khi module LoRa phát.

Một node IoT có nhiều loại tải khác nhau. Cảm biến thường tiêu thụ dòng nhỏ và yêu cầu nguồn ổn định. Relay và LED hồng ngoại có dòng tức thời cao hơn. LoRa khi phát cũng tạo dòng đỉnh. Nếu nguồn không đủ ổn định, node có thể reset, giá trị ADC dao động hoặc mất gói LoRa. Do đó khối nguồn cần được kiểm tra trước khi đánh giá các chức năng khác.



Hình 3.2: Luồng cấp nguồn chính của node

3.4.2 Mạch hạ áp 5V

Rail 5V cấp cho các cảm biến phổ biến, relay, transistor điều khiển và một số module ngoại vi. Nếu sử dụng nguồn đầu vào 24V, việc hạ áp bằng tuyến tính sẽ gây tổn hao nhiệt lớn. Vì vậy sơ đồ sử dụng khối hạ áp dạng switching để tạo 5V hiệu quả hơn. Tụ đầu vào và đầu ra của khối hạ áp có nhiệm vụ giảm ripple và ổn

định điện áp.

Đối với hệ thống có relay, dòng tải thay đổi theo trạng thái đóng cắt. Mỗi khi relay hút, cuộn dây tạo dòng đột biến; khi relay nhả, năng lượng trong cuộn dây có thể gây xung ngược. Mạch cần có diode dập xung tại cuộn relay và tụ lọc đủ gần rail 5V để tránh ảnh hưởng đến vi điều khiển.

3.4.3 Mạch ổn áp 3.3V

Module LoRa RA-02 hoạt động ở mức 3.3V. Sơ đồ sử dụng AMS1117-3.3V để tạo rail 3.3V từ 5V. AMS1117 là ổn áp tuyến tính dễ dùng, phù hợp với dòng tiêu thụ mức vừa phải. Tuy nhiên cần lưu ý nhiệt lượng nếu dòng 3.3V tăng cao. Trong thiết kế hiện tại, rail 3.3V chủ yếu cấp cho LoRa nên phương án này phù hợp.

Việc tách 5V và 3.3V giúp bảo vệ module LoRa khỏi mức logic không phù hợp. Khi Arduino Pro Mini chạy 5V, các chân SPI có thể tạo mức logic 5V. Vì RA-02 chỉ chịu 3.3V, thiết kế thực tế cần bảo đảm mức logic phù hợp, có thể thông qua lựa chọn Arduino Pro Mini 3.3V hoặc bổ sung mạch chuyển mức logic. Đây là điểm cần kiểm tra kỹ khi lắp mạch thật.

3.4.4 LED báo nguồn

Sơ đồ có các LED báo nguồn cho các rail như 5V, nguồn LoRa, nguồn module và LED trạng thái. LED báo nguồn giúp quá trình kiểm tra nhanh hơn: chỉ cần quan sát có thể biết rail nào đang hoạt động. Tuy nhiên điện trở hạn dòng cần được chọn phù hợp để tránh tiêu thụ dòng không cần thiết, đặc biệt nếu hệ thống hướng đến vận hành lâu dài bằng nguồn pin.

Rail nguồn	Phụ tải chính	Lưu ý kiểm thử
24V hoặc nguồn vào	Adapter, đầu cấp ngoài, tải mở rộng	Kiểm tra cực tính, điện áp không tải và điện áp khi relay đóng.
5V	Cảm biến, relay, transistor điều khiển, Arduino nếu bản 5V	Đo ripple khi relay hút và khi LoRa phát.

3.3V	LoRa RA-02, logic mức thấp	Không cấp nhầm 5V cho module LoRa; kiểm tra dòng đỉnh khi truyền.
GND	Toàn bộ hệ thống	Cần nối mass chung giữa nguồn, Arduino, cảm biến, relay và LoRa.

3.4.5 Rủi ro và biện pháp giảm thiểu

Các rủi ro chính của khối nguồn gồm sụt áp khi relay đóng, nhiễu ảnh hưởng đến ADC, nóng ỏn áp 3.3V và đấu nhầm nguồn. Biện pháp giảm rủi ro gồm dùng adapter đủ dòng, bố trí tụ lọc gần các module tiêu thụ dòng xung, đi dây mass hợp lý, tách đường công suất và đường tín hiệu analog, đồng thời kiểm tra điện áp từng rail trước khi gắn module nhạy như LoRa hoặc cảm biến pH.

Trong quá trình thử nghiệm, nên đo nguồn ở ba trạng thái: node chỉ đọc cảm biến, node truyền LoRa và node vừa truyền LoRa vừa đóng relay. Nếu điện áp không ổn định ở trạng thái thứ ba, cần tăng khả năng cấp dòng hoặc bổ sung tụ lọc trước khi đánh giá phần mềm.

3.5 Khối vi điều khiển Arduino Pro Mini

Arduino Pro Mini là trung tâm xử lý của node. Trên sơ đồ, Arduino Pro Mini được đặt ở khối vi điều khiển và kết nối ra các header để thuận tiện đấu dây, nạp chương trình và kiểm thử. Bo mạch này có kích thước nhỏ, dễ lập trình trong môi trường Arduino IDE hoặc PlatformIO, phù hợp với các ứng dụng cảm biến đơn giản.

3.5.1 Vai trò của vi điều khiển

Arduino Pro Mini thực hiện các nhiệm vụ sau:

- Khởi tạo cảm biến và ngoại vi.
- Đọc giá trị cảm biến theo chu kỳ.
- Lọc nhiễu và quy đổi dữ liệu sang đơn vị sử dụng.

- Đóng gói dữ liệu để gửi qua LoRa.
- Nhận lệnh điều khiển từ gateway.
- Điều khiển relay và bộ phát hồng ngoại.
- Gửi phản hồi trạng thái sau khi thực hiện lệnh.

Do tài nguyên hạn chế, chương trình trên Arduino cần được tổ chức theo vòng lặp trạng thái rõ ràng. Các thao tác đọc cảm biến, truyền LoRa và điều khiển thiết bị không nên chặn hệ thống quá lâu. Nếu sử dụng delay quá nhiều, node có thể phản hồi lệnh chậm hoặc bỏ lỡ gói tin từ gateway.

3.5.2 Bố trí header

Sơ đồ khối vi điều khiển có các header P1, P2 và P4 đưa các chân quan trọng ra ngoài. Việc sử dụng header giúp mạch linh hoạt hơn trong quá trình phát triển. Khi cần thay đổi loại cảm biến hoặc kiểm tra tín hiệu, người dùng có thể đo trực tiếp trên header mà không phải can thiệp vào mạch in.

Header	Nhóm chân	Chức năng
P1	RAW, GND, RST, VCC, DIO0, SCK, MISO, MOSI, NSS	Cấp nguồn, reset và giao tiếp SPI với LoRa.
P2	UITXo, UIRXo, GND, Po, A, TX, RL1, RL2	Giao tiếp UART, tín hiệu cảm biến analog, IR và relay.
P4	SCL, SDA, GND, A6, A7	Kết nối I2C và analog mở rộng.

3.5.3 Quản lý chân vào ra

Việc quản lý chân cần được cố định trong firmware bằng các hằng số rõ ràng. Ví dụ, chân RL1 điều khiển relay 1, RL2 điều khiển relay 2, TX điều khiển phát IR, Po đọc pH và A đọc cảm biến ánh sáng. Với cách đặt tên này, khi đọc mã nguồn có thể đối chiếu trực tiếp với sơ đồ nguyên lý.

Listing 3.1: Ví dụ định nghĩa chân trên node Arduino

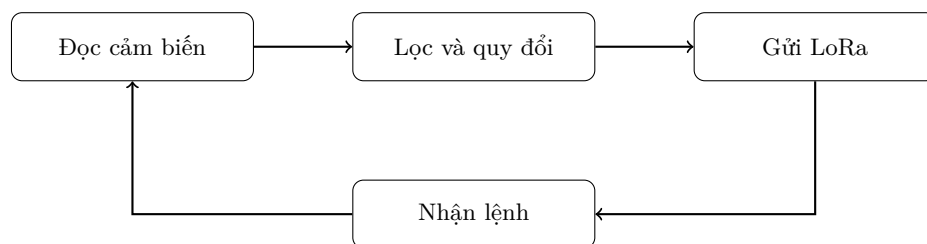
```
#define PIN_RELAY_1 7
#define PIN_RELAY_2 8
```

```
#define PIN_IR_TX 5
#define PIN_PH A1
#define PIN_LIGHT A0
#define PIN_TDS A2
#define PIN_LORA_NSS 10
#define PIN_LORA_RST 9
#define PIN_LORA_DIO0 2
```

Trong hệ thống thực tế, các chân phải được kiểm tra lại theo mạch in cuối cùng. Nếu sơ đồ nguyên lý, layout PCB và firmware không thống nhất, hệ thống có thể hoạt động sai dù từng khối riêng lẻ không lỗi. Vì vậy, bảng ánh xạ chân là một tài liệu bắt buộc đi kèm phần cứng.

3.5.4 Chu kỳ hoạt động của node

Chu kỳ hoạt động cơ bản của node gồm bốn pha. Pha thứ nhất là đọc cảm biến. Pha thứ hai là xử lý và đóng gói dữ liệu. Pha thứ ba là gửi dữ liệu qua LoRa. Pha thứ tư là lắng nghe lệnh điều khiển từ gateway trong một khoảng thời gian ngắn. Chu kỳ này lặp lại liên tục.



Hình 3.3: Chu kỳ hoạt động cơ bản của node

3.5.5 Quy tắc an toàn khi khởi động

Khi node vừa cấp nguồn, trạng thái relay cần được đặt về mức an toàn. Nếu relay điều khiển quạt hoặc đèn, hệ thống nên mặc định tắt thiết bị cho đến khi nhận được cấu hình hoặc lệnh hợp lệ. Với IR điều hòa, node không phát lệnh khi khởi động để tránh thay đổi trạng thái điều hòa ngoài ý muốn.

Trình tự khởi động đề xuất gồm: cấu hình chân output, đặt relay về OFF, khởi tạo Serial để debug, khởi tạo cảm biến, khởi tạo LoRa, gửi gói trạng thái đầu tiên và bắt đầu vòng lặp chính. Nếu LoRa không khởi tạo thành công, node vẫn có thể đọc cảm biến và báo lỗi qua Serial hoặc LED trạng thái.

3.6 Khối giao tiếp LoRa và RS485

Khối giao tiếp trong sơ đồ gồm module LoRa RA-02 và mạch MAX485. Trong phạm vi hệ thống hiện tại, LoRa là kênh truyền chính giữa node và gateway ESP32. MAX485 được xem như kênh mở rộng hoặc dự phòng cho các cảm biến công nghiệp, module ngoại vi hoặc đường truyền có dây trong trường hợp cần độ ổn định cao hơn.

3.6.1 Module LoRa RA-02

RA-02 là module LoRa sử dụng giao tiếp SPI. Các chân quan trọng gồm NSS, SCK, MOSI, MISO, RST và DI00. Module hoạt động ở mức 3.3V, vì vậy nguồn cấp và mức logic phải được kiểm tra cẩn thận. Trong sơ đồ, các tín hiệu SPI được đưa ra header riêng, giúp quá trình đo và sửa lỗi thuận tiện.

LoRa phù hợp với hệ thống vì dữ liệu cảm biến có dung lượng nhỏ và không yêu cầu băng thông cao. Một gói tin chỉ gồm một số giá trị đo và trạng thái thiết bị, có thể truyền định kỳ sau vài giây hoặc vài chục giây. So với Wi-Fi, LoRa giúp node đơn giản hơn ở lớp mạng, giảm yêu cầu cấu hình và tăng khả năng đặt node ở vị trí xa router.

3.6.2 Định dạng gói tin LoRa

Gói tin cần ngắn gọn, dễ phân tích và có khả năng mở rộng. Định dạng dạng chuỗi phân tách bằng dấu phẩy để debug khi dùng Serial Monitor. Tuy nhiên, dạng nhị phân hoặc JSON rút gọn có thể hiệu quả hơn khi cần giảm thời gian truyền. Trong khóa luận, định dạng chuỗi được chọn để dễ kiểm thử.

Listing 3.2: Gói dữ liệu node gửi lên gateway

```
DATA,1,30.9,65.0,2655.0,0.66,698.86,7.1,0,0,0
```

Ý nghĩa các trường lần lượt là loại gói, mã node, nhiệt độ, độ ẩm, CO2, ánh sáng, TDS, pH, trạng thái relay 1, relay 2 và điều hòa. Gateway sau khi nhận sẽ chuyển các trường này thành telemetry trên ThingsBoard.

Listing 3.3: Gói lệnh gateway gửi xuống node

```
CMD,1,RL1,ON  
CMD,1,RL2,OFF  
CMD,1,AC,TEMP_UP
```

```
CMD,1,AC,TEMP_DOWN
```

Node chỉ thực hiện lệnh khi mã node khớp với địa chỉ của nó và cấu trúc gói tin hợp lệ. Sau khi thực hiện, node gửi lại gói phản hồi:

Listing 3.4: Gói phản hồi trạng thái từ node

```
ACK,1,RL1,ON  
ACK,1,AC,TEMP_UP
```

3.6.3 Cơ chế kiểm tra lỗi

Trong môi trường căn hộ, LoRa có thể chịu ảnh hưởng bởi vật cản, nhiễu điện từ hoặc khoảng cách. Vì vậy gói tin nên có cơ chế kiểm tra lỗi. Ở mức đơn giản, gateway có thể kiểm tra số trường, kiểu dữ liệu và khoảng giá trị hợp lý. Ở mức tốt hơn, gói tin có thể bổ sung checksum.

Listing 3.5: Ví dụ gói tin có checksum

```
DATA,1,30.9,65.0,2655.0,0.66,698.86,7.1,0,0,0,*5A
```

Checksum giúp phát hiện gói tin bị sai trong quá trình truyền. Nếu checksum không khớp, gateway bỏ qua gói tin và không cập nhật telemetry. Đối với lệnh điều khiển, gateway có thể gửi lại nếu không nhận được ACK trong thời gian chờ.

3.6.4 Mạch MAX485

MAX485 là IC chuyển đổi UART sang RS485. RS485 có ưu điểm truyền xa, chống nhiễu tốt và hỗ trợ kết nối nhiều thiết bị trên cùng bus. Trong sơ đồ, MAX485 có các tín hiệu điều khiển hướng truyền/nhận, đường A/B và điện trở kết thúc. Mặc dù hệ thống hiện tại dùng LoRa làm kênh chính, việc bố trí MAX485 giúp node có khả năng mở rộng trong tương lai.

Ví dụ, một cảm biến CO2 công nghiệp hoặc cảm biến môi trường Modbus RTU có thể kết nối qua RS485. Khi đó Arduino Pro Mini sử dụng UART để gửi truy vấn và nhận phản hồi. Tuy nhiên, vì Arduino Pro Mini có tài nguyên hạn chế, cần cân nhắc nếu vừa dùng UART debug, vừa dùng RS485 và vừa dùng LoRa. Trong phiên bản cơ bản, RS485 có thể để dự phòng.

Kênh	Ưu điểm	Hạn chế
------	---------	---------

LoRa	Không dây, phạm vi tốt trong căn hộ, phù hợp dữ liệu nhỏ	Băng thông thấp, cần kiểm soát thời gian phát và xác nhận lệnh.
RS485	Ổn định, chống nhiễu, phù hợp cảm biến công nghiệp	Cần dây dẫn, phức tạp hơn khi lắp đặt trong căn hộ.
Wi-Fi trực tiếp tại node	Kết nối Internet trực tiếp, băng thông cao	Tốn năng lượng hơn, phụ thuộc sóng Wi-Fi tại vị trí đặt node.

3.6.5 Kiểm thử khối giao tiếp

Để kiểm thử LoRa, trước hết cần kiểm tra nguồn 3.3V tại module, sau đó kiểm tra SPI bằng chương trình khởi tạo đơn giản. Nếu module không khởi tạo được, cần kiểm tra các chân **NSS**, **RST**, **DIO0**, **MOSI**, **MISO** và **SCK**. Sau khi khởi tạo thành công, thực hiện truyền gói test giữa node và gateway ở khoảng cách ngắn trước, sau đó tăng khoảng cách và thêm vật cản.

Với RS485, kiểm thử gồm đo mức nguồn 5V, kiểm tra tín hiệu DE/RE, kiểm tra đường A/B và thử truyền UART giữa hai thiết bị. Nếu sử dụng điện trở kết thúc, cần bảo đảm giá trị điện trở phù hợp với chiều dài đường dây và cấu hình bus.

3.7 Khối cảm biến và khối actuator

Khối cảm biến và khối actuator là hai phần trực tiếp tương tác với môi trường căn hộ. Khối cảm biến cung cấp dữ liệu cho hệ thống đánh giá chất lượng không khí và điều kiện chăm sóc cây. Khối actuator thực hiện các lệnh điều khiển nhằm cải thiện môi trường hoặc hỗ trợ người dùng.

3.7.1 Khối cảm biến

Sơ đồ khối cảm biến có các đầu nối cho nhiều loại tín hiệu. Cảm biến pH kết nối qua đầu Po; cảm biến ánh sáng có tín hiệu A; cảm biến TDS dùng ngõ analog riêng; các cảm biến nhiệt độ, độ ẩm hoặc CO2 có thể kết nối qua I2C, UART hoặc

chân digital tùy module. Ngoài ra, sơ đồ còn có các điện trở kéo lên cho I2C, giúp đường SCL và SDA hoạt động ổn định.

Việc đo pH và TDS liên quan trực tiếp đến trạng thái nước hoặc dung dịch dinh dưỡng cho cây. Dữ liệu này không phải là thông số chất lượng không khí, nhưng giúp hệ thống đánh giá điều kiện cây trồng. Khi cây được chăm sóc ổn định, cây có khả năng duy trì trạng thái sinh trưởng tốt hơn và hỗ trợ môi trường trong nhà ở mức bền vững hơn.

Cảm biến	Tín hiệu	Vai trò
TDS	Analog	Theo dõi tổng chất rắn hòa tan trong nước hoặc dung dịch chăm sóc cây.
pH	Analog Po	Đánh giá độ axit/kiềm của dung dịch, hỗ trợ chăm sóc cây.
Ánh sáng	Analog A hoặc digital	Xác định điều kiện chiếu sáng cho cây và không gian sinh hoạt.
Nhiệt độ	I2C/digital	Đánh giá tiện nghi nhiệt và điều kiện môi trường.
Độ ẩm	I2C/digital	Theo dõi độ ẩm không khí, hỗ trợ cảnh báo khô hoặc ẩm cao.
CO2	UART/I2C/analog tùy module	Đánh giá thông gió và mật độ người trong phòng.

3.7.2 Loại và hiệu chuẩn cảm biến

Các cảm biến analog dễ bị dao động do nhiễu nguồn, dây dài hoặc sai số ADC. Vì vậy firmware cần lấy mẫu nhiều lần và tính trung bình. Với pH và TDS, hiệu chuẩn là bắt buộc nếu muốn dữ liệu có ý nghĩa thực tế. pH cần hiệu chuẩn bằng dung dịch chuẩn, còn TDS cần hiệu chuẩn bằng dung dịch có nồng độ đã biết.

Ví dụ, giá trị pH có thể được tính từ điện áp đầu ra của mạch đo theo phương trình tuyến tính:

$$pH = a \times V_{pH} + b$$

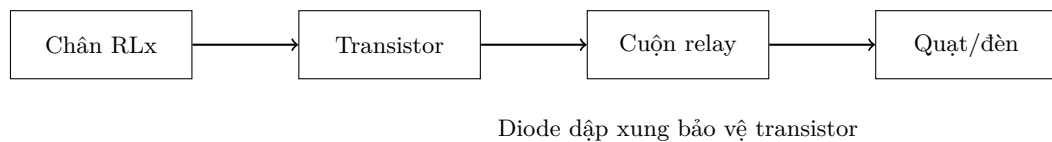
Trong đó a và b được xác định từ quá trình hiệu chuẩn. Tương tự, TDS có thể được tính từ điện áp sau khi bù nhiệt độ nếu cảm biến hỗ trợ. Trong khóa luận,

phần hiệu chuẩn được trình bày như một bước cần thực hiện khi đưa hệ thống vào sử dụng thực tế.

3.7.3 Khối relay

Khối actuator trong sơ đồ có ba relay. Mỗi relay được điều khiển thông qua transistor, có điện trở base/gate và diode dập xung cuộn dây. Cách thiết kế này giúp Arduino không phải cấp dòng trực tiếp cho relay. Khi chân điều khiển ở mức kích hoạt, transistor dẫn, cuộn relay có dòng và tiếp điểm chuyển trạng thái.

Relay có thể điều khiển quạt, đèn hoặc tải ngoài. Trong ứng dụng hiện tại, RL1 và RL2 được hiển thị trên app ở tab Actuator, trạng thái ban đầu là OFF. Người dùng có thể bật/tắt relay từ ứng dụng Flutter. Node nhận lệnh qua LoRa, điều khiển relay tương ứng và gửi ACK về gateway.



Hình 3.4: Nguyên lý điều khiển relay của node

3.7.4 Khối phát hồng ngoại

Khối IR gồm các LED hồng ngoại và transistor điều khiển. Arduino tạo chuỗi xung điều chế tương ứng với mã điều khiển điều hòa. Transistor giúp cấp dòng đủ lớn cho LED IR để tăng khoảng cách phát. Trên app, thiết bị điều hòa được hiển thị như một actuator riêng. Các lệnh chính gồm bật/tắt, tăng nhiệt độ và giảm nhiệt độ.

Điều khiển IR có một hạn chế quan trọng: node không biết chắc điều hòa đã nhận được lệnh hay chưa, trừ khi có cảm biến phản hồi hoặc camera/IR receiver kiểm tra. Vì vậy hệ thống lưu trạng thái dự đoán dựa trên lệnh đã gửi. Nếu người dùng dùng remote gốc của điều hòa, trạng thái trên app có thể lệch với thực tế. Đây là điểm cần nêu rõ trong phần thảo luận.

3.7.5 Mô hình trạng thái thiết bị

Để quản lý actuator, node lưu trạng thái cục bộ cho từng thiết bị. Mỗi relay có hai trạng thái ON và OFF. Điều hòa có thể có trạng thái OFF, ON, nhiệt độ đặt và chế độ điều khiển. Trong phiên bản cơ bản, app hiển thị điều hòa ở mức ON/OFF và cho phép tăng/giảm nhiệt độ.

Thiết bị	Lệnh hỗ trợ	Phản hồi từ node
RL1	ON, OFF	Trạng thái relay sau khi điều khiển.
RL2	ON, OFF	Trạng thái relay sau khi điều khiển.
Điều hòa	ON, OFF, TEMP_UP, TEMP_DOWN	Trạng thái dự đoán và lệnh IR đã phát.

3.7.6 Kiểm thử khối cảm biến và actuator

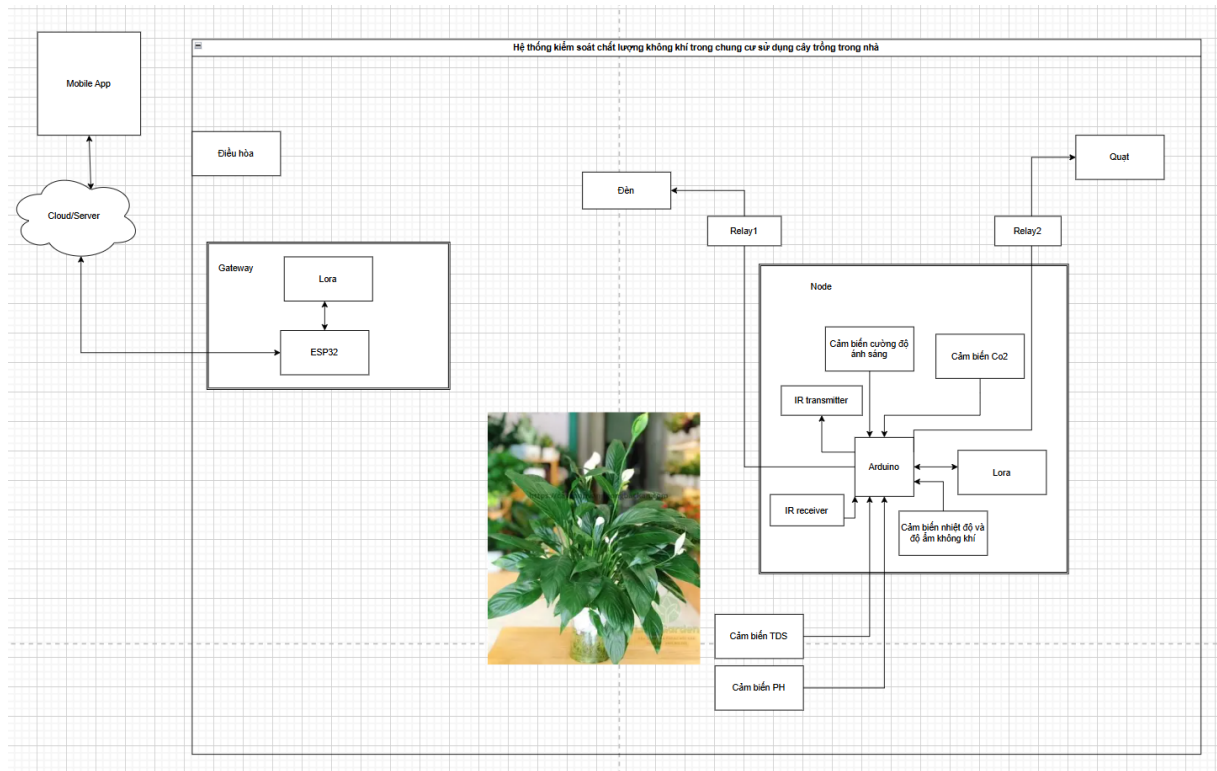
Kiểm thử cảm biến cần thực hiện theo từng cảm biến riêng lẻ trước khi tích hợp. Với cảm biến ánh sáng, có thể che cảm biến hoặc chiếu đèn để quan sát giá trị thay đổi. Với nhiệt độ và độ ẩm, có thể so sánh với thiết bị đo tham chiếu. Với CO2, cần kiểm tra phản ứng khi phòng kín hoặc khi thổi khí gần cảm biến, nhưng vẫn phải chú ý an toàn và giới hạn của module. Với pH và TDS, cần dùng dung dịch chuẩn để đánh giá.

Kiểm thử actuator cần bắt đầu khi chưa đấu tải AC nguy hiểm. Trước hết đo tín hiệu điều khiển ở chân Arduino, sau đó kiểm tra transistor và relay bằng tải DC nhỏ hoặc đồng hồ đo thông mạch. Khi relay hoạt động đúng mới đấu tải thực tế. Với IR, có thể dùng camera điện thoại để quan sát LED hồng ngoại nhấp nháy khi phát lệnh, sau đó thử với điều hòa thật.

THIẾT KẾ PHẦN MỀM VÀ TRIỂN KHAI HỆ THỐNG

Chương này trình bày thiết kế và triển khai hệ thống kiểm soát chất lượng không khí trong căn hộ chung cư sử dụng cây trồng trong nhà. Hệ thống được thiết kế theo kiến trúc nhiều lớp gồm lớp cảm biến - chấp hành, lớp truyền thông LoRa, lớp gateway, lớp nền tảng IoT và lớp ứng dụng người dùng. Cách tổ chức này kế thừa hướng tiếp cận của các hệ thống IAQ tích hợp IoT: đo thông số môi trường theo thời gian thực, phân tích dữ liệu trên gateway hoặc cloud, sau đó điều khiển thiết bị nhằm duy trì điều kiện phù hợp cho con người và cây trồng.

4.1 Kiến trúc tổng thể



Hình 4.1: Kiến trúc tổng thể của hệ thống

Hệ thống gồm bốn khối chính:

- **Node cảm biến và điều khiển:** sử dụng Arduino Pro Mini để đọc TDS, pH, nhiệt độ, độ ẩm, CO2, ánh sáng và điều khiển relay, IR. Node đóng vai trò điểm đo tại khu vực cây trồng hoặc không gian sinh hoạt.
- **Kênh truyền LoRa:** truyền dữ liệu giữa node và gateway bằng hai module LoRa.
- **Gateway ESP32:** nhận dữ liệu từ node, gửi telemetry lên ThingsBoard và chuyển lệnh điều khiển từ ThingsBoard về node.
- **ThingsBoard và app Flutter:** lưu trữ, hiển thị dữ liệu, cảnh báo và cung cấp giao diện điều khiển cho người dùng.

Dữ liệu môi trường được thu thập định kỳ tại node. Sau khi đọc cảm biến, node chuẩn hóa dữ liệu, đóng gói và gửi qua LoRa. Gateway ESP32 giải mã gói tin, kiểm tra mã node, kiểm tra checksum nếu có và gửi dữ liệu lên ThingsBoard.

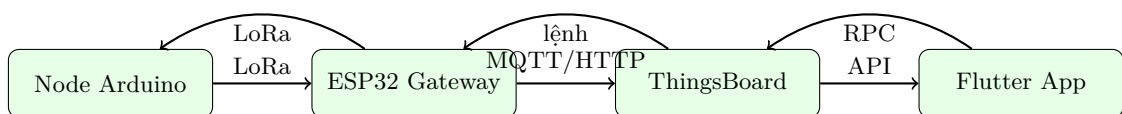
Khi người dùng điều khiển quạt, đèn hoặc điều hòa trên app Flutter, lệnh được gửi đến ThingsBoard, gateway nhận lệnh và truyền xuống node qua LoRa. Nhờ đó, hệ thống hình thành vòng kín ở mức ứng dụng: đo lường, hiển thị, ra quyết định và phản hồi trạng thái.

4.2 Luồng dữ liệu tổng quát

Hệ thống được tổ chức theo hai luồng chính: luồng giám sát và luồng điều khiển. Luồng giám sát đi từ node lên app, còn luồng điều khiển đi từ app xuống node. Việc tách hai luồng này giúp thiết kế phần mềm rõ ràng và dễ kiểm thử.

Trong luồng giám sát, node đọc cảm biến theo chu kỳ. Các thông số gồm TDS, pH, ánh sáng, CO₂, nhiệt độ và độ ẩm được đóng gói thành bản tin LoRa. Gateway nhận bản tin, chuyển đổi sang JSON và gửi lên ThingsBoard. App Flutter truy vấn dữ liệu từ ThingsBoard để hiển thị trên tab Sensor. Ảnh giao diện Sensor cho thấy các thông số được trình bày dưới dạng thẻ, mỗi thẻ có biểu tượng, tên đại lượng và giá trị hiện tại. Cách bố trí này giúp người dùng đọc nhanh trạng thái node trên màn hình điện thoại.

Trong luồng điều khiển, người dùng chuyển sang tab Actuator để điều khiển RL1, RL2 và điều hòa. Khi nhấn một thiết bị, app gửi lệnh đến ThingsBoard. Gateway nhận lệnh này và chuyển thành bản tin LoRa gửi tới node. Node thực thi lệnh, cập nhật trạng thái relay hoặc phát mã IR, sau đó gửi ACK về gateway. Gateway cập nhật lại trạng thái lên ThingsBoard để app hiển thị đúng.



Hình 4.2: Luồng dữ liệu hai chiều của hệ thống

4.3 Các chế độ vận hành

Hệ thống có hai chế độ vận hành: tự động và thủ công. Trên giao diện app, công tắc *Manual* thể hiện chế độ điều khiển bằng tay. Khi Manual được bật, người dùng chủ động bật/tắt relay và điều khiển điều hòa. Khi Manual tắt, hệ thống có thể áp dụng các luật điều khiển tự động dựa trên ngưỡng.

Chế độ thủ công phù hợp trong giai đoạn thử nghiệm vì người dùng có thể kiểm tra ngay phản hồi của từng actuator. Ví dụ, khi nhấn RL1, relay 1 phải đổi trạng thái; khi nhấn điều hòa, node phải phát mã IR. Chế độ tự động phù hợp khi hệ thống đã được hiệu chuẩn và người dùng đã xác định ngưỡng vận hành mong muốn. Các ngưỡng này có thể bao gồm bật quạt khi CO2 cao, bật đèn khi ánh sáng thấp, cảnh báo khi pH/TDS vượt vùng phù hợp hoặc gợi ý mở cửa khi phòng thiếu thông gió.

Chế độ	Nguồn quyết định	Ví dụ hành vi
Manual	Người dùng trên app	Bật RL1, tắt RL2, tăng/giảm nhiệt độ điều hòa theo thao tác trực tiếp.
Auto	Luật điều khiển trong gateway hoặc ThingsBoard	Tự bật quạt khi CO2 cao, bật đèn khi ánh sáng thấp, cảnh báo khi pH/TDS bất thường.

4.4 Thiết kế node cảm biến

Node cảm biến được xây dựng xung quanh Arduino Pro Mini. Các cảm biến TDS và pH được sử dụng để theo dõi chất lượng nước hoặc dung dịch chăm sóc cây. Cảm biến nhiệt độ, độ ẩm và CO2 phục vụ đánh giá chất lượng không khí trong căn hộ. Cảm biến ánh sáng giúp xác định điều kiện chiếu sáng cho cây và không gian sinh hoạt.

4.4.1 Khối đọc cảm biến

Chương trình trên node thực hiện chu kỳ đọc dữ liệu theo các bước: khởi tạo cảm biến, lấy mẫu nhiều lần, lọc nhiễu đơn giản bằng trung bình cộng, quy đổi ra đơn vị đo và lưu vào cấu trúc dữ liệu. Với cảm biến analog, giá trị ADC được chuyển đổi theo hệ số hiệu chuẩn. Với cảm biến digital, node đọc dữ liệu theo giao thức của module và kiểm tra lỗi trước khi sử dụng.

4.4.2 Khối điều khiển

Node điều khiển hai relay để bật/tắt quạt và đèn. Relay được điều khiển qua chân GPIO, có trạng thái mặc định an toàn khi node khởi động. Bộ phát IR được

dùng để gửi mã tăng hoặc giảm nhiệt độ điều hòa. Khi nhận lệnh IR từ gateway, node phát chuỗi mã tương ứng và gửi phản hồi trạng thái để gateway cập nhật lên ThingsBoard.

4.4.3 Định dạng gói tin node gửi lên gateway

Gói tin uplink được thiết kế ngắn gọn để phù hợp với LoRa:

NODE_ID,T,H,CO2,LUX,TDS,PH,FAN,LIGHT

Trong đó T là nhiệt độ, H là độ ẩm, CO2 là nồng độ CO2, LUX là ánh sáng, TDS là tổng chất rắn hòa tan, PH là chỉ số pH, FAN và LIGHT là trạng thái relay.

4.4.4 Thuật toán đọc cảm biến

Để giảm nhiễu, node không sử dụng trực tiếp một mẫu ADC đơn lẻ. Với các kênh analog, node lấy nhiều mẫu liên tiếp, loại bỏ một số mẫu bất thường nếu cần và tính trung bình. Với cảm biến digital, node kiểm tra mã lỗi hoặc điều kiện timeout trước khi cập nhật giá trị. Nếu một cảm biến lỗi trong một chu kỳ, node có thể giữ giá trị gần nhất và đánh dấu trạng thái lỗi để gateway không hiểu nhầm là dữ liệu mới hợp lệ.

Listing 4.1: Mã giả đọc và lọc cảm biến analog

```
float readAnalogAverage(uint8_t pin) {
    long sum = 0;
    for (int i = 0; i < 20; i++) {
        sum += analogRead(pin);
        delay(5);
    }
    float adc = sum / 20.0;
    return adc * 5.0 / 1023.0;
}
```

Sau khi có điện áp đo, firmware quy đổi thành giá trị vật lý. Các hệ số quy đổi cần được lưu trong cấu hình để thuận tiện hiệu chuẩn. Đối với pH, có thể dùng hai điểm chuẩn pH 4 và pH 7 hoặc pH 7 và pH 10. Đối với TDS, cần dùng dung dịch chuẩn và bù nhiệt độ nếu yêu cầu độ chính xác cao.

4.4.5 Quản lý trạng thái node

Node không chỉ gửi dữ liệu cảm biến mà còn gửi trạng thái thiết bị. Các trạng thái gồm RL1, RL2 và điều hòa. Việc gửi kèm trạng thái giúp app hiển thị đúng sau khi người dùng điều khiển hoặc sau khi node khởi động lại. Nếu app chỉ lưu trạng thái cục bộ, khi mất kết nối hoặc reset node, giao diện có thể không phản ánh đúng trạng thái thực tế.

Listing 4.2: Cấu trúc dữ liệu trạng thái node

```
struct NodeState {  
    float temperature;  
    float humidity;  
    float co2;  
    float lux;  
    float tds;  
    float ph;  
    bool relay1;  
    bool relay2;  
    bool acPower;  
    int acSetpoint;  
};
```

4.4.6 Xử lý lệnh điều khiển tại node

Node xử lý lệnh theo nguyên tắc kiểm tra trước khi thực thi. Bản tin phải đúng loại CMD, đúng mã node và đúng tên thiết bị. Nếu lệnh không hợp lệ, node bỏ qua và không thay đổi trạng thái. Với relay, node đổi mức GPIO. Với điều hòa, node gọi hàm phát mã IR tương ứng.

Listing 4.3: Mã giả xử lý lệnh actuator

```
void handleCommand(Command cmd) {  
    if (cmd.nodeId != NODE_ID) return;  
  
    if (cmd.device == "RL1") {  
        setRelay(PIN_RELAY_1, cmd.action == "ON");  
    } else if (cmd.device == "RL2") {  
        setRelay(PIN_RELAY_2, cmd.action == "ON");  
    } else if (cmd.device == "AC") {  
        sendIrCommand(cmd.action);  
    }  
  
    sendAck(cmd.device, cmd.action);  
}
```

4.5 Thiết kế gateway ESP32 và ThingsBoard

Gateway ESP32 là cầu nối giữa mạng LoRa cục bộ và Internet. ESP32 giao tiếp với module LoRa qua SPI hoặc UART tùy module sử dụng. Khi nhận gói tin từ node, gateway tách chuỗi dữ liệu, kiểm tra số trường, chuyển đổi kiểu dữ liệu và tạo payload gửi lên ThingsBoard.

4.5.1 Kết nối ThingsBoard

Gateway được cấu hình token thiết bị của ThingsBoard. Dữ liệu telemetry được gửi theo dạng JSON, trong đó mỗi khóa biểu diễn một thông số đo như `temperature`, `humidity`, `co2`, `lux`, `tds` và `ph`. Trên ThingsBoard, dashboard hiển thị các widget gồm đồng hồ đo, biểu đồ thời gian, trạng thái quạt/đèn và cảnh báo khi thông số vượt ngưỡng. Các ngưỡng có thể được đặt theo nhu cầu sử dụng, chẳng hạn CO2 cao, ánh sáng thấp hoặc pH nằm ngoài khoảng phù hợp cho cây.

4.5.2 Mô hình thiết bị trên ThingsBoard

Trong ThingsBoard, gateway hoặc node được đại diện bởi một thiết bị có access token riêng. Telemetry được gửi lên theo từng khóa. Cách đặt tên khóa cần thống nhất với app Flutter để tránh lỗi khi truy vấn dữ liệu. Các khóa đề xuất gồm `temperature`, `humidity`, `co2`, `lux`, `tds`, `ph`, `r11`, `r12`, `acPower` và `acSetpoint`.

Telemetry	Đơn vị	Ý nghĩa
<code>temperature</code>	°C	Nhiệt độ không khí tại khu vực node.
<code>humidity</code>	%	Độ ẩm tương đối của không khí.
<code>co2</code>	ppm	Nồng độ CO2, dùng đánh giá mức thông gió.
<code>lux</code>	lux	Cường độ ánh sáng tại vị trí cây.
<code>tds</code>	ppm	Tổng chất rắn hòa tan trong nước hoặc dung dịch.

ph	không đơn vị	Độ pH của dung dịch chăm sóc cây.
r11, r12	ON/OFF	Trạng thái relay.
acPower	ON/OFF	Trạng thái dự đoán của điều hòa.

4.5.3 Rule chain và cảnh báo

ThingsBoard có thể xử lý dữ liệu thông qua rule chain. Khi telemetry mới được gửi lên, rule chain kiểm tra ngưỡng và tạo cảnh báo nếu cần. Ví dụ, nếu CO2 vượt ngưỡng, hệ thống có thể tạo alarm "CO2 cao"; nếu ánh sáng thấp, hệ thống tạo alarm "Thiếu sáng"; nếu pH nằm ngoài khoảng cho phép, hệ thống tạo alarm "pH bất thường".

Các cảnh báo này giúp người dùng không cần quan sát app liên tục. Trong phiên bản mở rộng, ThingsBoard có thể gửi notification hoặc app Flutter có thể lấy danh sách alarm để hiển thị nổi bật. Đây là điểm quan trọng vì hệ thống kiểm soát chất lượng không khí cần phản ứng với tình huống bất thường, không chỉ hiển thị dữ liệu.

4.5.4 Luồng điều khiển xuống node

ThingsBoard cung cấp cơ chế RPC hoặc shared attributes để gửi lệnh xuống gateway. Gateway lắng nghe lệnh, chuyển thành gói LoRa dạng:

`CMD,NODE_ID,DEVICE,ACTION`

Ví dụ `CMD,1,FAN,ON` để bật quạt, `CMD,1,LIGHT,OFF` để tắt đèn, `CMD,1,AC,TEMP_UP` để tăng nhiệt độ điều hòa và `CMD,1,AC,TEMP_DOWN` để giảm nhiệt độ điều hòa. Node nhận lệnh, thực thi và phản hồi trạng thái.

4.5.5 Xử lý mất kết nối

Gateway cần xử lý hai loại mất kết nối: mất LoRa với node và mất Internet với ThingsBoard. Khi mất LoRa, gateway không nhận được dữ liệu mới và có thể đánh dấu node offline sau một khoảng timeout. Khi mất Internet, gateway vẫn có

thể nhận dữ liệu LoRa nhưng không gửi được lên ThingsBoard. Trong trường hợp này, gateway nên có bộ đệm cục bộ để lưu tạm dữ liệu.

Listing 4.4: Mã giả xử lý gửi telemetry tại gateway

```
if (wifiConnected() && thingsboardConnected()) {  
    sendTelemetry(payload);  
} else {  
    cacheTelemetry(payload);  
}  
  
if (connectionRecovered()) {  
    flushCachedTelemetry();  
}
```

4.6 Thiết kế ứng dụng Flutter

Ứng dụng Flutter được thiết kế để người dùng theo dõi trạng thái căn hộ và điều khiển thiết bị từ điện thoại. Giao diện chính hiển thị các thông số nhiệt độ, độ ẩm, CO₂, ánh sáng, TDS và pH. Mỗi thông số có màu trạng thái để người dùng nhanh chóng nhận biết bình thường, cảnh báo hoặc vượt ngưỡng.

Ứng dụng có các màn hình chức năng chính:

- **Màn hình tổng quan:** hiển thị giá trị mới nhất và trạng thái môi trường.
- **Màn hình biểu đồ:** hiển thị lịch sử thay đổi của từng thông số theo thời gian.
- **Màn hình điều khiển:** bật/tắt quạt, bật/tắt đèn, tăng/giảm nhiệt độ điều hòa.
- **Màn hình cấu hình:** thiết lập ngưỡng cảnh báo cho CO₂, nhiệt độ, độ ẩm, ánh sáng, TDS và pH.

Ứng dụng không giao tiếp trực tiếp với node mà kết nối với ThingsBoard. Cách thiết kế này giúp người dùng có thể truy cập từ xa, đồng thời giảm độ phức tạp của node và gateway. Mọi dữ liệu và lệnh đều được đồng bộ qua nền tảng IoT, bảo đảm hệ thống có thể mở rộng khi bổ sung thêm node trong tương lai.

4.6.1 Màn hình đăng nhập

Ảnh giao diện đăng nhập cho thấy ứng dụng có tên hiển thị *Smart Farm*, biểu tượng lá cây và hai trường nhập email, password. Cách đặt tên này phù hợp với hướng hệ thống chăm sóc cây thông minh, tuy nhiên trong phạm vi khóa luận có thể hiểu app là giao diện điều khiển node cây trồng trong nhà và chất lượng không khí. Màn hình đăng nhập giúp giới hạn quyền truy cập, tránh người ngoài điều khiển relay hoặc điều hòa.



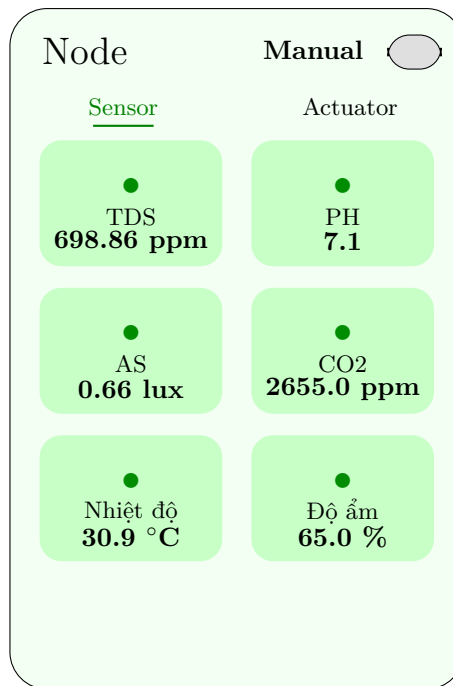
Hình 4.3: Minh họa màn hình đăng nhập của ứng dụng

4.6.2 Màn hình Sensor

Màn hình Sensor hiển thị sáu thẻ dữ liệu gồm TDS, pH, AS, CO₂, nhiệt độ và độ ẩm. Dữ liệu trong ảnh lần lượt là TDS 698.86 ppm, pH 7.1, ánh sáng 0.66 lux, CO₂ 2655 ppm, nhiệt độ 30.9 °C và độ ẩm 65%. Đây là nhóm thông số phản ánh đồng thời tình trạng cây và chất lượng không khí. CO₂ ở mức 2655 ppm cho thấy phòng có thể đang thiếu thông gió; ánh sáng 0.66 lux cho thấy vị trí cây gần như thiếu sáng; nhiệt độ 30.9 °C tương đối cao đối với không gian sinh hoạt.

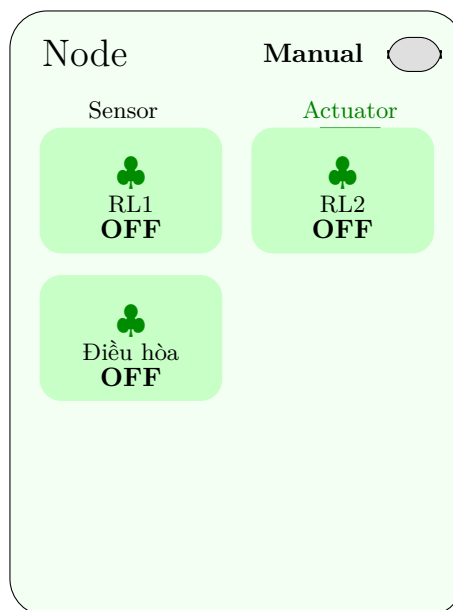
4.6.3 Màn hình Actuator

Màn hình Actuator hiển thị ba thiết bị gồm RL1, RL2 và điều hòa. Trong ảnh, cả ba thiết bị đều ở trạng thái OFF. Bố cục dạng thẻ đồng nhất với màn



Hình 4.4: Minh họa màn hình Sensor hiển thị dữ liệu node

hình Sensor, giúp người dùng dễ chuyển đổi giữa giám sát và điều khiển. Khi người dùng nhấn vào một thẻ actuator, app gửi lệnh tương ứng đến ThingsBoard.



Hình 4.5: Minh họa màn hình Actuator điều khiển relay và điều hòa

4.6.4 Ánh xạ dữ liệu từ ThingsBoard sang giao diện

App Flutter cần ánh xạ telemetry từ ThingsBoard sang các thành phần giao diện. Mỗi thẻ Sensor tương ứng với một khóa telemetry. Nếu khóa không tồn tại hoặc dữ liệu quá cũ, app nên hiển thị trạng thái mất dữ liệu thay vì giữ nguyên giá trị cũ mà không cảnh báo. Điều này đặc biệt quan trọng với CO2 và nhiệt độ vì người dùng có thể đưa ra quyết định dựa trên dữ liệu hiển thị.

Thẻ giao diện	Telemetry	Cách hiển thị
TDS	tds	Hiển thị ppm, cảnh báo khi vượt ngưỡng cây.
PH	ph	Hiển thị một chữ số thập phân hoặc hai chữ số tùy hiệu chuẩn.
AS	lux	Hiển thị lux, cảnh báo thiếu sáng.
CO2	co2	Hiển thị ppm, cảnh báo khi phòng thiếu thông gió.
Nhiệt độ	temperature	Hiển thị °C, phục vụ điều khiển điều hòa.
Độ ẩm	humidity	Hiển thị %, cảnh báo khô hoặc ẩm cao.

4.6.5 Bảo mật và quyền điều khiển

Vì app có khả năng điều khiển thiết bị thật, bảo mật không thể bỏ qua. Màn hình đăng nhập giúp xác thực người dùng trước khi truy cập dữ liệu. Token ThingsBoard không nên hard-code trực tiếp trong mã nguồn public. App cần lưu token đăng nhập trong bộ nhớ an toàn của thiết bị và xóa token khi người dùng đăng xuất.

Ngoài xác thực, hệ thống cần phân quyền. Người dùng thông thường có thể xem dữ liệu và điều khiển thủ công; tài khoản quản trị có thể thay đổi ngưỡng, cấu hình node hoặc thêm thiết bị. Phân quyền rõ ràng giúp hạn chế rủi ro khi nhiều người cùng sử dụng hệ thống trong một căn hộ.

KẾT QUẢ VÀ THẢO LUẬN

Chương này trình bày kết quả kiểm thử hệ thống và thảo luận về khả năng đáp ứng các mục tiêu đã đặt ra. Do hệ thống gồm nhiều khối phần cứng và phần mềm, quá trình đánh giá được thực hiện theo từng khối trước khi kiểm thử tích hợp toàn bộ.

5.1 Kịch bản kiểm thử

Các kịch bản kiểm thử được xây dựng dựa trên môi trường căn hộ chung cư. Node cảm biến được đặt gần khu vực cây trồng trong nhà, gateway ESP32 đặt tại vị trí có kết nối Wi-Fi ổn định. Khoảng cách giữa node và gateway được thay đổi trong phạm vi các phòng khác nhau để đánh giá khả năng truyền LoRa.

Các nhóm kiểm thử gồm:

- Kiểm thử đọc dữ liệu từ từng cảm biến.
- Kiểm thử truyền gói LoRa từ node đến gateway.
- Kiểm thử gateway gửi telemetry lên ThingsBoard.
- Kiểm thử app Flutter hiển thị dữ liệu từ ThingsBoard.
- Kiểm thử lệnh điều khiển relay và IR từ app xuống node.

5.2 Kết quả thu thập dữ liệu cảm biến

Node Arduino Pro Mini đọc được đầy đủ các thông số TDS, pH, nhiệt độ, độ ẩm, CO2 và ánh sáng. Với các cảm biến analog, việc lấy mẫu nhiều lần và tính

trung bình giúp giảm dao động tức thời của ADC. Dữ liệu sau khi quy đổi được đóng gói theo định dạng thống nhất và truyền về gateway.

Trong quá trình thử nghiệm, nhiệt độ và độ ẩm phản ứng rõ khi bật/tắt điều hòa hoặc quạt. Giá trị CO₂ tăng khi phòng đóng kín và có người ở trong thời gian dài, sau đó giảm khi tăng thông gió. Cảm biến ánh sáng phản ánh được sự thay đổi giữa ban ngày, ban đêm và khi bật đèn bổ sung. TDS và pH cung cấp thông tin phục vụ theo dõi dung dịch chăm sóc cây.

5.3 Kết quả truyền thông LoRa và gateway

Liên kết LoRa giữa node và gateway hoạt động ổn định với các gói dữ liệu có kích thước nhỏ. Gateway ESP32 nhận gói tin, tách trường dữ liệu và gửi lên ThingsBoard theo đúng cấu trúc telemetry. Khi Wi-Fi của gateway hoạt động ổn định, dữ liệu từ node xuất hiện trên dashboard ThingsBoard gần như theo thời gian thực.

Trong trường hợp gateway mất kết nối Internet tạm thời, dữ liệu không thể gửi lên ThingsBoard nhưng kênh LoRa giữa node và gateway vẫn hoạt động. Điều này cho thấy hệ thống có thể tách biệt vấn đề truyền thông cục bộ và vấn đề kết nối Internet. Để tăng độ tin cậy trong phiên bản sau, gateway có thể bổ sung bộ đệm dữ liệu cục bộ và gửi bù khi Internet khôi phục.

5.4 Kết quả hiển thị và điều khiển trên ứng dụng

Ứng dụng Flutter hiển thị được giá trị mới nhất của các thông số môi trường thông qua dữ liệu lấy từ ThingsBoard. Các nút điều khiển quạt, đèn và điều hòa được gửi về ThingsBoard, sau đó gateway chuyển tiếp xuống node qua LoRa.

Khi người dùng bật quạt hoặc đèn trên ứng dụng, relay tại node thay đổi trạng thái tương ứng. Khi người dùng nhấn tăng hoặc giảm nhiệt độ điều hòa, node phát mã IR đã cấu hình. Cơ chế phản hồi trạng thái giúp ứng dụng cập nhật lại trạng thái thiết bị sau khi lệnh được thực thi, tránh tình trạng giao diện hiển thị khác với thiết bị thực tế.

5.5 Thảo luận

Kết quả thử nghiệm cho thấy kiến trúc đề xuất phù hợp với bài toán giám sát và điều khiển môi trường trong căn hộ. So với hướng chỉ dùng thiết bị lọc không khí hoặc chỉ giám sát cây trồng, hệ thống kết hợp cả dữ liệu IAQ và dữ liệu chăm sóc cây, từ đó hỗ trợ người dùng nhìn nhận môi trường sống theo một bức tranh đầy đủ hơn. Việc sử dụng LoRa cho kết nối node - gateway giúp node không phụ thuộc trực tiếp vào Wi-Fi, phù hợp với vị trí đặt cây hoặc khu vực sóng Wi-Fi yếu. ThingsBoard giúp rút ngắn thời gian phát triển nhờ có sẵn chức năng lưu trữ telemetry, dashboard và điều khiển từ xa. Flutter phù hợp để xây dựng ứng dụng di động có giao diện trực quan và dễ mở rộng.

Một số hạn chế vẫn cần được cải thiện. Thứ nhất, độ chính xác của cảm biến TDS, pH và CO₂ phụ thuộc nhiều vào hiệu chuẩn; đây cũng là thách thức thường gặp trong các hệ thống IoT môi trường. Thứ hai, điều khiển điều hòa bằng IR là điều khiển một chiều; node cần có cơ chế lưu trạng thái dự đoán hoặc bổ sung cảm biến phản hồi để biết chắc điều hòa đã nhận lệnh. Thứ ba, hệ thống mới triển khai với một node nên chưa đánh giá đầy đủ khả năng mở rộng nhiều node. Thứ tư, đề tài chưa đo trực tiếp VOC hoặc bụi mịn, nên chưa thể kết luận định lượng đầy đủ về khả năng lọc không khí của cây. Các hạn chế này là cơ sở cho hướng phát triển tiếp theo.

KẾT LUẬN

Những công việc đã thực hiện trong khóa luận

Khóa luận đã nghiên cứu và phát triển một nguyên mẫu hệ thống kiểm soát chất lượng không khí trong căn hộ chung cư có tích hợp cây lan ý và công nghệ IoT. Trên cơ sở tham khảo các hướng tiếp cận về IAQ, cây trồng/thủy canh và hệ thống giám sát thông minh, đề tài xây dựng mô hình phù hợp với bối cảnh căn hộ: quy mô nhỏ, chi phí hợp lý, dễ lắp đặt, có khả năng theo dõi dữ liệu theo thời gian thực và hỗ trợ điều khiển thiết bị. Hệ thống được tổ chức theo kiến trúc IoT gồm node cảm biến Arduino Pro Mini, liên kết LoRa, gateway ESP32, nền tảng ThingsBoard và ứng dụng Flutter.

Các kết quả chính đã đạt được gồm:

- Thiết kế node cảm biến đọc được các thông số TDS, pH, nhiệt độ, độ ẩm, CO₂ và ánh sáng.
- Xây dựng cơ chế truyền dữ liệu không dây giữa node và gateway bằng hai module LoRa.
- Phát triển gateway ESP32 để nhận dữ liệu LoRa, xử lý gói tin và gửi dữ liệu lên ThingsBoard.
- Thiết kế mô hình dữ liệu trên ThingsBoard phục vụ hiển thị, cảnh báo và điều khiển.
- Xây dựng ứng dụng Flutter để người dùng theo dõi thông số môi trường và điều khiển thiết bị.
- Triển khai điều khiển relay bật/tắt quạt, relay bật/tắt đèn và điều khiển điều hòa bằng IR tại node.

Kết quả triển khai cho thấy hệ thống có thể giám sát môi trường trong nhà, hỗ trợ chăm sóc cây lan ý và thực hiện điều khiển thiết bị từ xa. Kiến trúc node

- gateway - cloud - app phù hợp với điều kiện căn hộ chung cư và có thể mở rộng khi bổ sung thêm node cảm biến. Tuy nhiên, để đánh giá hiệu quả cải thiện IAQ ở mức định lượng, hệ thống cần được kiểm thử trong thời gian dài hơn, với cảm biến được hiệu chuẩn, nhiều điều kiện thông gió khác nhau và các thông số bổ sung như VOC hoặc bụi mịn. **Hướng phát triển tương lai**

Trong các phiên bản tiếp theo, hệ thống có thể được phát triển theo các hướng sau:

- Hiệu chuẩn cảm biến TDS, pH và CO₂ bằng mẫu chuẩn để tăng độ chính xác của dữ liệu.
- Bổ sung cơ chế lưu đệm tại gateway khi mất Internet và tự động gửi bù dữ liệu sau khi kết nối phục hồi.
- Mở rộng hệ thống nhiều node cho nhiều phòng hoặc nhiều khu vực cây trồng trong căn hộ.
- Bổ sung thuật toán điều khiển tự động dựa trên ngưỡng hoặc luật điều khiển, ví dụ tự bật quạt khi CO₂ cao hoặc tự bật đèn khi ánh sáng thấp.
- Bổ sung cảm biến VOC, bụi mịn và lưu lượng thông gió để đánh giá IAQ đầy đủ hơn.
- Thử nghiệm với nhiều loài cây trong nhà hoặc mô hình thủy canh nhỏ để so sánh khả năng duy trì điều kiện sinh trưởng và hỗ trợ cải thiện môi trường.
- Cải thiện điều khiển điều hòa bằng cách lưu trạng thái thiết bị, học mã IR của nhiều hãng điều hòa và bổ sung phản hồi nhiệt độ sau điều khiển.
- Nâng cấp ứng dụng Flutter với thông báo đẩy, biểu đồ lịch sử chi tiết và cấu hình ngưỡng riêng cho từng loại cây.

Nhìn chung, đề tài đã chứng minh tính khả thi của việc kết hợp cây trồng trong nhà với hệ thống IoT để giám sát và kiểm soát chất lượng không khí trong căn hộ. Hướng tiếp cận này có thể tiếp tục mở rộng thành giải pháp nhà thông minh hướng tới sức khỏe, tiết kiệm năng lượng, chăm sóc cây trồng tự động và quản lý môi trường sống bền vững.

PHỤ LỤC THIẾT KẾ FIRMWARE VÀ GÓI TIN

A.1 Mục tiêu của firmware node

Firmware node có nhiệm vụ biến mạch phần cứng thành một thiết bị IoT hoàn chỉnh. Chương trình phải đọc cảm biến, xử lý dữ liệu, truyền LoRa, nhận lệnh, điều khiển actuator và phản hồi trạng thái. Vì node sử dụng Arduino Pro Mini, thiết kế firmware cần đơn giản, tiết kiệm bộ nhớ và tránh các thao tác gây treo vòng lặp chính.

Các mục tiêu cụ thể gồm:

- Khởi tạo ổn định tất cả ngoại vi sau khi cấp nguồn.
- Đọc các cảm biến TDS, pH, ánh sáng, nhiệt độ, độ ẩm và CO2 theo chu kỳ.
- Lọc dữ liệu analog để giảm nhiễu tức thời.
- Đóng gói dữ liệu theo định dạng thống nhất.
- Gửi dữ liệu qua LoRa đến gateway.
- Nhận gói lệnh từ gateway và điều khiển RL1, RL2, điều hòa.
- Gửi ACK để gateway và app cập nhật trạng thái.

A.2 Cấu trúc chương trình node

Firmware được chia thành các module logic. Cách chia module giúp mã nguồn dễ đọc và dễ sửa khi thay đổi phần cứng. Trong triển khai đơn giản trên Arduino, các module có thể nằm trong cùng một file nhưng vẫn được tách bằng nhóm hàm.

Module	Hàm chính	Vai trò
Khởi tạo	<code>setupPins()</code> , <code>setupSensors()</code> , <code>setupLoRa()</code>	Cấu hình GPIO, cảm biến và module truyền thông.
Cảm biến	<code>readTds()</code> , <code>readPh()</code> , <code>readEnvironment()</code>	Thu thập và quy đổi giá trị đo.
LoRa	<code>sendTelemetry()</code> , <code>receiveCommand()</code>	Gửi dữ liệu và nhận lệnh.
Actuator	<code>setRelay()</code> , <code>sendIrCommand()</code>	Điều khiển relay và điều hòa.
Trạng thái	<code>updateState()</code> , <code>buildPayload()</code>	Lưu trạng thái node và tạo gói tin.

A.3 Trình tự khởi động

Khi node được cấp nguồn, firmware phải đưa các actuator về trạng thái an toàn trước khi khởi tạo những phần khác. Nếu relay bị kích ngoài ý muốn trong giai đoạn khởi động, quạt hoặc đèn có thể bật không kiểm soát. Vì vậy các chân relay cần được cấu hình output và ghi mức OFF ngay trong những dòng đầu của hàm `setup()`.

Listing A.1: Trình tự khởi động đề xuất cho node

```
void setup() {
    setupPins();
    setRelay(PIN_RELAY_1, false);
    setRelay(PIN_RELAY_2, false);

    Serial.begin(9600);
    setupSensors();
    setupLoRa();

    state.relay1 = false;
    state.relay2 = false;
    state.acPower = false;
    state.acSetpoint = 26;

    sendBootMessage();
}
```

Sau khi khởi động, node gửi một bản tin boot về gateway. Bản tin này giúp gateway biết node vừa reset hoặc vừa được cấp nguồn. Nếu node reset bất thường nhiều lần, gateway có thể ghi log để phục vụ chẩn đoán lỗi nguồn hoặc lỗi firmware.

A.4 Vòng lặp chính

Vòng lặp chính cần cân bằng giữa đọc cảm biến và phản hồi lệnh. Nếu chỉ đọc cảm biến rồi delay dài, node sẽ phản hồi chậm khi người dùng điều khiển trên app. Nếu lắng nghe LoRa liên tục mà không đọc cảm biến đúng chu kỳ, dữ liệu telemetry sẽ thiếu ổn định. Một giải pháp là sử dụng bộ đếm thời gian dựa trên `millis()` thay vì `delay()` dài.

Listing A.2: Vòng lặp chính không chặn

```
unsigned long lastSensorRead = 0;
unsigned long lastTelemetrySend = 0;

void loop() {
    unsigned long now = millis();

    if (now - lastSensorRead >= SENSOR_INTERVAL_MS) {
        readAllSensors();
        lastSensorRead = now;
    }

    if (now - lastTelemetrySend >= TELEMETRY_INTERVAL_MS) {
        sendTelemetry();
        lastTelemetrySend = now;
    }

    if (receiveCommand()) {
        handleCommand(currentCommand);
    }
}
```

Với cách này, node có thể vừa gửi dữ liệu định kỳ vừa phản hồi lệnh nhanh. Thời gian đọc cảm biến vẫn cần được kiểm soát, đặc biệt với những cảm biến có thời gian đáp ứng chậm như CO2. Nếu cảm biến CO2 dùng UART và cần thời gian khởi động dài, firmware nên đánh dấu trạng thái warming-up thay vì gửi giá trị chưa ổn định.

A.5 Định dạng dữ liệu telemetry

Gói telemetry cần đủ thông tin nhưng không quá dài. Một gói đề xuất:

Listing A.3: Định dạng gói telemetry

```
DATA,<nodeId>,<temperature>,<humidity>,<co2>,<lux>,<tds>,<ph>,<r11>,<r12>,<ac>,<checksum>
```

Ví dụ:

```
DATA,1,30.9,65.0,2655.0,0.66,698.86,7.10,0,0,0,5A
```

Trường	Kiểu	Mô tả
DATA	Chuỗi	Loại gói tin.
nodeId	Số nguyên	Mã định danh của node.
temperature	Số thực	Nhiệt độ đo được, đơn vị °C.
humidity	Số thực	Độ ẩm tương đối, đơn vị %.
co2	Số thực	Nồng độ CO2, đơn vị ppm.
lux	Số thực	Cường độ ánh sáng, đơn vị lux.
tds	Số thực	Tổng chất rắn hòa tan, đơn vị ppm.
ph	Số thực	Chỉ số pH.
r11, r12, ac	0/1	Trạng thái thiết bị.
checksum	Hex	Mã kiểm tra lỗi đơn giản.

A.6 Định dạng gói điều khiển

Gói điều khiển từ gateway xuống node có dạng:

```
CMD,<nodeId>,<device>,<action>,<checksum>
```

Ví dụ:

```
CMD,1,RL1,ON,2F
CMD,1,RL2,OFF,31
CMD,1,AC,TEMP_UP,7B
CMD,1,AC,TEMP_DOWN,7C
```

Node cần kiểm tra `nodeId`. Nếu lệnh không dành cho node hiện tại, node bỏ qua. Nếu `device` không thuộc danh sách hỗ trợ, node phản hồi lỗi. Nếu checksum sai, node không thực thi để tránh hành vi ngoài ý muốn.

A.7 Gói phản hồi ACK và ERR

Sau khi thực thi lệnh, node gửi ACK:

```
ACK,1,RL1,ON
```

Nếu lệnh lỗi, node gửi ERR:

```
ERR,1,BAD_CHECKSUM  
ERR,1,UNKNOWN_DEVICE  
ERR,1,UNKNOWN_ACTION
```

Gateway sử dụng ACK để cập nhật trạng thái lên ThingsBoard. Nếu không nhận ACK sau một khoảng thời gian, gateway có thể gửi lại lệnh hoặc báo lỗi trên app. Điều này giúp người dùng biết lệnh có được thực hiện hay không.

A.8 Tính checksum

Checksum đơn giản có thể tính bằng XOR tất cả ký tự trước dấu checksum. Phương pháp này không mạnh như CRC nhưng đủ để phát hiện nhiều lỗi truyền cơ bản trong gói tin ngắn.

Listing A.4: Ví dụ tính checksum XOR

```
uint8_t calcChecksum(const char *payload) {  
    uint8_t cs = 0;  
    for (int i = 0; payload[i] != '\0'; i++) {  
        cs ^= payload[i];  
    }  
    return cs;  
}
```

Khi đóng gói, firmware chuyển checksum sang dạng hex hai ký tự. Khi nhận, node hoặc gateway tính lại checksum và so sánh với trường cuối cùng. Nếu không khớp, gói tin bị loại bỏ.

A.9 Hiệu chuẩn pH

Cảm biến pH cần hiệu chuẩn vì điện áp đầu ra phụ thuộc vào module đo, đầu dò và nhiệt độ. Quy trình hiệu chuẩn hai điểm:

1. Rửa đầu dò bằng nước sạch và lau nhẹ.

2. Đặt đầu dò vào dung dịch chuẩn pH 7, chờ giá trị ổn định.
3. Ghi lại điện áp V_7 .
4. Rửa đầu dò và đặt vào dung dịch chuẩn pH 4 hoặc pH 10.
5. Ghi lại điện áp V_4 hoặc V_{10} .
6. Tính hệ số tuyến tính a và b .

Nếu dùng hai điểm pH 7 và pH 4:

$$a = \frac{7 - 4}{V_7 - V_4}$$

$$b = 7 - a \times V_7$$

Sau đó:

$$pH = a \times V_{pH} + b$$

A.10 Hiệu chuẩn TDS

TDS cũng cần hiệu chuẩn bằng dung dịch chuẩn. Quy trình:

1. Chuẩn bị dung dịch TDS có giá trị đã biết.
2. Đặt đầu dò vào dung dịch, chờ ổn định.
3. Ghi điện áp đầu ra.
4. Điều chỉnh hệ số chuyển đổi trong firmware.
5. Kiểm tra lại với ít nhất một mẫu khác nếu có.

TDS chịu ảnh hưởng bởi nhiệt độ. Nếu cần chính xác hơn, firmware có thể bù nhiệt độ:

$$TDS_{comp} = \frac{TDS}{1 + \alpha(T - 25)}$$

Trong đó α là hệ số bù nhiệt độ và T là nhiệt độ dung dịch.

A.11 Xử lý dữ liệu CO2

Cảm biến CO2 thường cần thời gian khởi động để giá trị ổn định. Firmware nên bỏ qua một số mẫu đầu sau khi cấp nguồn. Nếu cảm biến hỗ trợ trạng thái

sẵn sàng, node chỉ gửi giá trị CO2 khi trạng thái hợp lệ. Nếu không, node có thể đặt cờ `co2Valid`.

Listing A.5: Kiểm tra dữ liệu CO2 trước khi gửi

```
if (co2SensorReady()) {  
    state.co2 = readCo2();  
    state.co2Valid = true;  
} else {  
    state.co2Valid = false;  
}
```

Gateway có thể dùng cờ hợp lệ để quyết định có cập nhật telemetry hay không. Điều này tránh trường hợp dashboard hiển thị giá trị 0 ppm hoặc giá trị sai trong giai đoạn cảm biến chưa ổn định.

A.12 Điều khiển IR cho điều hòa

Điều khiển điều hòa bằng IR cần lưu mã lệnh của từng chức năng. Mỗi hãng điều hòa có bộ mã khác nhau. Node cần có bảng mã tương ứng với thiết bị thực tế. Trong phiên bản cơ bản, các lệnh cần hỗ trợ gồm bật/tắt, tăng nhiệt độ và giảm nhiệt độ.

Listing A.6: Khung hàm phát lệnh IR

```
void sendIrCommand(String action) {  
    if (action == "POWER") {  
        irsend.sendRaw(IR_POWER, IR_POWER_LEN, 38);  
    } else if (action == "TEMP_UP") {  
        irsend.sendRaw(IR_TEMP_UP, IR_TEMP_UP_LEN, 38);  
        state.acSetpoint++;  
    } else if (action == "TEMP_DOWN") {  
        irsend.sendRaw(IR_TEMP_DOWN, IR_TEMP_DOWN_LEN, 38);  
        state.acSetpoint--;  
    }  
}
```

Vì IR là điều khiển một chiều, trạng thái điều hòa trong hệ thống là trạng thái dự đoán. Khi người dùng dùng remote gốc, trạng thái dự đoán có thể sai. Để khắc phục, có thể bổ sung cảm biến nhiệt độ phản hồi hoặc cơ chế đồng bộ lại bằng thao tác trên app.

A.13 Firmware gateway

Gateway ESP32 cần thực hiện ba nhóm nhiệm vụ: nhận LoRa, gửi ThingsBoard và xử lý lệnh xuống node. Gateway có tài nguyên mạnh hơn Arduino Pro Mini nên có thể đảm nhiệm logic phức tạp hơn như lưu đệm dữ liệu, reconnect Wi-Fi và quản lý timeout.

Listing A.7: Luồng xử lý chính của gateway

```
void loop() {
    maintainWifi();
    maintainThingsBoard();

    if (loraPacketAvailable()) {
        String packet = readLoraPacket();
        if (validateTelemetry(packet)) {
            JsonDocument doc = convertToJson(packet);
            sendToThingsBoard(doc);
        }
    }

    if (thingsBoardRpcAvailable()) {
        Command cmd = readRpcCommand();
        sendCommandToNode(cmd);
    }
}
```

A.14 Quản lý reconnect

Kết nối Wi-Fi có thể mất tạm thời. Gateway cần reconnect tự động và không được treo chương trình khi mất mạng. Khi mất ThingsBoard, gateway vẫn nên tiếp tục nhận LoRa và lưu dữ liệu quan trọng.

Sự cố	Cách phát hiện	Xử lý đề xuất
Mất Wi-Fi	WiFi.status() khác connected	Thử reconnect theo chu kỳ, không chặn vòng lặp.
Mất ThingsBoard	MQTT/HTTP lỗi hoặc timeout	Tạo lại kết nối, lưu đệm telemetry.
Mất LoRa node	Không nhận gói trong thời gian timeout	Đánh dấu node offline trên ThingsBoard.

Lệnh không ACK	Không nhận ACK sau timeout	Gửi lại số lần giới hạn và báo lỗi app.
----------------	----------------------------	---

A.15 Quy ước log

Log giúp debug nhanh trong giai đoạn phát triển. Node và gateway nên thống nhất các mức log:

- INFO: hoạt động bình thường như boot, gửi telemetry.
- WARN: dữ liệu cảm biến bất thường, mất ACK lần đầu.
- ERROR: lỗi khởi tạo LoRa, lỗi kết nối ThingsBoard kéo dài.

Listing A.8: Ví dụ log gateway

```
[INFO] WiFi connected
[INFO] ThingsBoard connected
[INFO] RX LoRa: DATA,1,30.9,65.0,2655.0,0.66,698.86,7.10,0,0,0
[WARN] Node 1 CO2 high: 2655 ppm
[ERROR] RPC command timeout: RL1 ON
```

A.16 Bảng cấu hình đề xuất

Các tham số cấu hình nên được gom ở đầu chương trình hoặc lưu trong file cấu hình nếu dùng ESP32.

Tham số	Giá trị đề xuất	Ý nghĩa
SENSOR_INTERVAL_MS	2000	Chu kỳ đọc cảm biến.
TELEMETRY_INTERVAL_MS	1000	Chu kỳ gửi telemetry.
COMMAND_TIMEOUT_MS	3000	Thời gian chờ ACK.
MAX_RETRY	3	Số lần gửi lại lệnh.
CO2_HIGH	1000 ppm	Ngưỡng cảnh báo CO2 cao.
TEMP_HIGH	30 °C	Ngưỡng nhiệt độ cao.
LIGHT_LOW	100 lux	Ngưỡng thiếu sáng tham khảo.

PHỤ LỤC KIỂM THỬ VÀ HƯỚNG DẪN VẬN HÀNH

B.1 Mục tiêu kiểm thử

Kiểm thử nhằm xác nhận hệ thống hoạt động đúng theo yêu cầu thiết kế. Do hệ thống gồm phần cứng, firmware, gateway, nền tảng ThingsBoard và app Flutter, kiểm thử cần thực hiện theo từng lớp. Nếu chỉ kiểm thử toàn hệ thống ở cuối, việc xác định nguyên nhân lỗi sẽ khó khăn hơn.

Các mục tiêu kiểm thử gồm:

- Xác nhận các rail nguồn ổn định.
- Xác nhận Arduino Pro Mini đọc đúng cảm biến.
- Xác nhận LoRa truyền dữ liệu giữa node và gateway.
- Xác nhận ESP32 gửi telemetry lên ThingsBoard.
- Xác nhận app Flutter hiển thị đúng dữ liệu.
- Xác nhận lệnh điều khiển từ app đến relay và IR.
- Xác nhận hệ thống xử lý được mất kết nối tạm thời.

B.2 Kiểm thử khối nguồn

Kiểm thử nguồn cần thực hiện trước khi cắm các module nhạy như LoRa hoặc cảm biến pH. Dùng đồng hồ đo điện áp tại các rail 24V, 5V và 3.3V. Sau đó kiểm tra lại khi relay đóng để phát hiện sụt áp.

Mã test	Điều kiện	Giá trị kỳ vọng	Kết luận
---------	-----------	-----------------	----------

PWR-01	Cấp nguồn đầu vào, chưa gắn tải	5V và 3.3V ổn định	Đạt nếu sai số nhỏ.
PWR-02	Relay RL1 hút	5V không sụt gây reset	Đạt nếu Arduino không reset.
PWR-03	LoRa phát liên tục	3.3V ổn định	Đạt nếu LoRa không mất kết nối.
PWR-04	Relay và LoRa hoạt động đồng thời	Không reset node	Đạt nếu telemetry vẫn gửi đều.

B.3 Kiểm thử cảm biến

Mỗi cảm biến được kiểm tra riêng trước khi kiểm thử tích hợp. Giá trị đo không nhất thiết phải chính xác tuyệt đối trong giai đoạn đầu, nhưng phải thay đổi hợp lý khi điều kiện môi trường thay đổi.

Cảm biến	Cách thử	Kỳ vọng
Nhiệt độ	Đặt gần nguồn nhiệt nhẹ hoặc thay đổi điều hòa	Giá trị tăng/giảm theo môi trường.
Độ ẩm	So sánh với thiết bị đo tham chiếu	Sai số nằm trong mức chấp nhận của module.
CO2	Đặt trong phòng kín và phòng thông thoáng	Giá trị tăng khi phòng kín, giảm khi thông gió.
Ánh sáng	Che cảm biến và chiếu đèn	Giá trị lux giảm/tăng rõ ràng.
TDS	Đo nước sạch và dung dịch có khoáng	Dung dịch khoáng có TDS cao hơn.
pH	Đo dung dịch chuẩn hoặc mẫu khác nhau	Giá trị thay đổi theo tính axit/kiềm.

B.4 Kiểm thử LoRa

Kiểm thử LoRa gồm truyền gói test, truyền telemetry thật và truyền lệnh điều khiển. Ban đầu đặt node và gateway gần nhau để loại trừ vấn đề khoảng cách. Sau đó tăng khoảng cách và đặt vật cản để mô phỏng căn hộ.

Mã test	Kịch bản	Tiêu chí đạt
LORA-01	Gửi gói HELLO từ node đến gateway	Gateway nhận đúng nội dung.
LORA-02	Gửi telemetry 100 lần ở khoảng cách gần	Tỷ lệ nhận cao, không sai định dạng.
LORA-03	Gửi telemetry qua một lớp tường	Gateway vẫn nhận ổn định.
LORA-04	Gateway gửi lệnh RL1 ON xuống node	Node bật relay và gửi ACK.
LORA-05	Gửi lệnh sai checksum	Node bỏ qua lệnh.

B.5 Kiểm thử ThingsBoard

Sau khi gateway nhận telemetry, bước tiếp theo là gửi dữ liệu lên ThingsBoard. Kiểm thử cần xác nhận đúng token thiết bị, đúng endpoint và đúng tên telemetry.

Mã test	Kịch bản	Kỳ vọng
TB-01	Gateway gửi JSON telemetry mẫu	ThingsBoard hiển thị dữ liệu mới.
TB-02	Gửi đủ sáu thông số cảm biến	Dashboard cập nhật tất cả widget.
TB-03	CO2 vượt ngưỡng	Rule chain tạo cảnh báo.
TB-04	App gửi RPC bật RL1	Gateway nhận được lệnh RPC.
TB-05	Gateway mất mạng tạm thời	Không treo chương trình, reconnect sau đó.

B.6 Kiểm thử app Flutter

App Flutter được kiểm thử theo các màn hình trong ảnh: đăng nhập, Sensor và Actuator. Màn hình đăng nhập kiểm tra xác thực. Màn hình Sensor kiểm tra hiển thị telemetry. Màn hình Actuator kiểm tra gửi lệnh.

Mã test	Kịch bản	Tiêu chí đạt
APP-01	Nhập email/password đúng	Vào được màn hình Node.
APP-02	Nhập sai mật khẩu	App báo lỗi, không vào hệ thống.
APP-03	Mở tab Sensor	Hiển thị TDS, pH, AS, CO2, nhiệt độ, độ ẩm.
APP-04	Mở tab Actuator	Hiển thị RL1, RL2 và điều hòa.
APP-05	Bật Manual và nhấn RL1	Lệnh được gửi đến ThingsBoard.
APP-06	Node phản hồi ACK	Trạng thái trên app cập nhật.

B.7 Kiểm thử actuator

Actuator cần được kiểm thử thận trọng vì liên quan đến thiết bị điện. Trong giai đoạn đầu, chỉ dùng tải giả hoặc đo thông mạch relay. Khi mạch điều khiển ổn định mới đấu quạt, đèn hoặc điều hòa thật.

Mã test	Kịch bản	Kỳ vọng
ACT-01	Gửi lệnh RL1 ON	Relay 1 hút, app hiển thị ON.
ACT-02	Gửi lệnh RL1 OFF	Relay 1 nhả, app hiển thị OFF.
ACT-03	Gửi lệnh RL2 ON/OFF	Relay 2 thay đổi đúng.
ACT-04	Gửi lệnh AC TEMP_UP	LED IR phát mã tăng nhiệt độ.

ACT-05	Gửi lệnh AC TEMP_DOWN	LED IR phát mã giảm nhiệt độ.
ACT-06	Gửi lệnh không hợp lệ	Node không thay đổi actuator.

B.8 Kịch bản vận hành trong căn hộ

Kịch bản vận hành mô phỏng một ngày sử dụng hệ thống:

1. Buổi sáng, người dùng mở app kiểm tra ánh sáng và độ ẩm.
2. Nếu ánh sáng thấp, người dùng bật đèn qua RL2 hoặc hệ thống tự bật khi ở chế độ Auto.
3. Khi phòng có người trong thời gian dài, CO2 tăng. App hiển thị cảnh báo.
4. Người dùng bật quạt qua RL1 để tăng thông gió.
5. Khi nhiệt độ tăng cao, người dùng dùng lệnh IR để giảm nhiệt độ điều hòa.
6. Cuối ngày, người dùng kiểm tra TDS và pH để quyết định thay nước hoặc bổ sung dung dịch chăm sóc cây.

B.9 Phân tích dữ liệu mẫu từ ảnh app

Ảnh tab Sensor cho thấy:

- TDS = 698.86 ppm.
- pH = 7.1.
- Ánh sáng = 0.66 lux.
- CO2 = 2655 ppm.
- Nhiệt độ = 30.9 °C.
- Độ ẩm = 65.0%.

Từ các giá trị này có thể đưa ra nhận xét: CO2 cao cho thấy phòng cần thông gió; ánh sáng rất thấp nên cây có thể thiếu sáng; nhiệt độ 30.9 °C khá cao nên cần

điều hòa hoặc quạt; độ ẩm 65% vẫn ở mức có thể chấp nhận nhưng cần theo dõi nếu kéo dài; pH 7.1 gần trung tính; TDS 698.86 ppm cần so sánh với loại cây cụ thể để đánh giá phù hợp hay không.

B.10 Bảng ngưỡng cảnh báo đề xuất

Các ngưỡng dưới đây chỉ là tham khảo ban đầu và cần điều chỉnh theo loại cây, thói quen sử dụng căn hộ và cảm biến thực tế.

Thông số	Ngưỡng cảnh báo	Hành động đề xuất
CO ₂	> 1000 ppm	Bật quạt, mở cửa hoặc tăng thông gió.
CO ₂	> 2000 ppm	Cảnh báo mức cao, ưu tiên thông gió ngay.
Nhiệt độ	> 30°C	Bật quạt hoặc giảm nhiệt độ điều hòa.
Độ ẩm	< 40%	Cảnh báo khô, cân nhắc tăng ẩm.
Độ ẩm	> 75%	Cảnh báo ẩm cao, tăng thông gió.
Ánh sáng	< 100 lux	Bật đèn bổ sung cho cây nếu cần.
pH	ngoài khoảng 5.5–7.5	Kiểm tra dung dịch chăm sóc cây.
TDS	tùy loại cây	Điều chỉnh dung dịch dinh dưỡng.

B.11 Hướng dẫn vận hành

Quy trình vận hành cơ bản:

1. Cấp nguồn cho gateway ESP32 và kiểm tra gateway kết nối Wi-Fi.
2. Cấp nguồn cho node Arduino Pro Mini.
3. Chờ node gửi bản tin boot và telemetry đầu tiên.
4. Mở app Flutter và đăng nhập.
5. Kiểm tra tab Sensor để xác nhận dữ liệu cập nhật.

6. Chuyển sang tab Actuator để kiểm thử RL1, RL2 và điều hòa nếu cần.
7. Theo dõi dashboard ThingsBoard để kiểm tra lịch sử dữ liệu.

B.12 Bảo trì hệ thống

Hệ thống cần được bảo trì định kỳ để bảo đảm dữ liệu đáng tin cậy. Đầu dò pH cần được vệ sinh và bảo quản đúng cách. Cảm biến TDS cần được rửa sau khi đo dung dịch có nồng độ cao. Cảm biến CO2 cần tránh bụi và hơi nước. Module LoRa và gateway cần được đặt ở vị trí ít bị che chắn kim loại.

Thành phần	Chu kỳ đề xuất	Công việc
Đầu dò pH	2–4 tuần	Vệ sinh, hiệu chuẩn lại bằng dung dịch chuẩn.
Cảm biến TDS	1 tháng	Kiểm tra với dung dịch chuẩn hoặc mẫu tham chiếu.
Cảm biến CO2	3–6 tháng	Kiểm tra sai lệch và vệ sinh khu vực cảm biến.
Relay	Khi thay tải	Kiểm tra tiếp điểm, tải định mức và dây đấu.
Gateway	Khi đổi Wi-Fi	Cập nhật SSID, mật khẩu và token ThingsBoard.
App	Khi đổi API/token	Kiểm tra đăng nhập và quyền điều khiển.

B.13 Danh sách lỗi thường gặp

Hiện tượng	Nguyên nhân có thể	Cách xử lý
App không có dữ liệu	Gateway mất Wi-Fi, sai token, node chưa gửi LoRa	Kiểm tra gateway, ThingsBoard và log LoRa.
CO2 luôn bằng 0	Cảm biến chưa khởi động, sai giao tiếp, lỗi nguồn	Kiểm tra thời gian warm-up và dây tín hiệu.

pH dao động mạnh	Nhiều analog, đầu dò bẩn, chưa hiệu chuẩn	Lọc mẫu, vệ sinh đầu dò, hiệu chuẩn lại.
Relay không bật	Sai chân GPIO, transistor lỗi, thiếu nguồn relay	Đo tín hiệu chân RL và điện áp cuộn relay.
Điều hòa không nhận IR	Sai mã IR, LED đặt sai hướng, dòng LED yếu	Học lại mã IR, kiểm tra camera, tăng dòng phù hợp.
LoRa chập chờn	Anten kém, nguồn 3.3V yếu, khoảng cách/vật cản lớn	Kiểm tra anten, nguồn và vị trí đặt gateway.

B.14 Tiêu chí nghiệm thu

Hệ thống được xem là đạt yêu cầu nếu thỏa mãn các tiêu chí:

- Node gửi đủ sáu thông số cảm biến lên ThingsBoard.
- App hiển thị đúng dữ liệu mới nhất trên tab Sensor.
- App điều khiển được RL1 và RL2 từ tab Actuator.
- Node phát được lệnh IR cho điều hòa.
- Gateway tự reconnect khi Wi-Fi mất tạm thời.
- Hệ thống có cảnh báo khi CO₂, nhiệt độ, ánh sáng hoặc pH vượt ngưỡng.

B.15 Đề xuất cải tiến sau kiểm thử

Sau quá trình kiểm thử, các cải tiến có giá trị gồm thêm vỏ bảo vệ cho node, bổ sung đầu nối chống cắm ngược, thêm cầu chì hoặc bảo vệ quá dòng cho tải relay, bổ sung lưu đệm dữ liệu trên gateway, hoàn thiện thông báo đẩy trên app và xây dựng chức năng cấu hình ngưỡng trực tiếp từ Flutter.

Đối với điều khiển điều hòa, nên bổ sung thư viện hỗ trợ nhiều mã IR và giao diện chọn hãng điều hòa. Đối với cảm biến cây trồng, nên lưu cấu hình theo từng loại cây, ví dụ cây ưa sáng, cây chịu bóng, cây thủy sinh hoặc cây trồng đất. Khi

đó hệ thống không chỉ hiển thị giá trị đo mà còn đưa ra khuyến nghị phù hợp với cây cụ thể.

B.16 Mẫu nhật ký kiểm thử

Nhật ký kiểm thử giúp lưu lại điều kiện đo, phiên bản firmware, vị trí đặt node và kết quả quan sát. Nếu hệ thống có lỗi sau khi thay đổi phần cứng hoặc phần mềm, nhật ký giúp truy vết nhanh hơn. Mỗi lần kiểm thử nên ghi rõ ngày giờ, người thực hiện, phiên bản firmware node, phiên bản firmware gateway và phiên bản app.

Thời điểm	Cấu hình	Kết quả	Ghi chú
Ngày 1 - sáng	Node gần gateway, không vật cản	Telemetry ổn định	Dùng để kiểm tra baseline.
Ngày 1 - chiều	Node cách gateway một phòng	Telemetry ổn định	Tỷ lệ nhận gói cần ghi bằng log.
Ngày 2 - sáng	Bật/tắt RL1 20 lần	Relay phản hồi đúng	Kiểm tra nhiệt độ transistor.
Ngày 2 - chiều	Gửi lệnh IR đến điều hòa	Điều hòa nhận lệnh	Cần đặt LED IR đúng hướng.
Ngày 3 - tối	Phòng kín 30 phút	CO2 tăng rõ	Kiểm tra cảnh báo trên app.

Khi ghi nhật ký, cần phân biệt giữa lỗi phần cứng và lỗi phần mềm. Ví dụ, nếu relay không bật nhưng chân Arduino có thay đổi mức logic, lỗi có thể nằm ở transistor, relay hoặc nguồn relay. Nếu chân Arduino không đổi mức, lỗi có thể nằm ở firmware hoặc lệnh từ gateway. Cách phân tích này giúp tiết kiệm thời gian sửa lỗi.

B.17 Phiếu hiệu chuẩn cảm biến pH

Phiếu hiệu chuẩn pH được dùng để ghi lại điện áp đọc từ cảm biến tại các dung dịch chuẩn. Dữ liệu này phục vụ tính hệ số tuyến tính trong firmware. Nếu

thay đầu dò pH, thay module khuếch đại hoặc thay nguồn cấp, cần hiệu chuẩn lại.

Dung dịch chuẩn	Điện áp đo	Giá trị ADC	Ghi chú
pH 4.00			Chờ ổn định trước khi ghi.
pH 7.00			Rửa đầu dò trước khi đo.
pH 10.00			Dùng nếu cần hiệu chuẩn ba điểm.
Mẫu thực tế 1			So sánh sau hiệu chuẩn.
Mẫu thực tế 2			Kiểm tra độ lặp lại.

Sau khi có dữ liệu, tính hệ số và cập nhật firmware. Cần ghi lại hệ số đã sử dụng để lần sau có thể so sánh. Nếu sau một thời gian giá trị pH lệch nhiều so với dung dịch chuẩn, đầu dò có thể bị bẩn, khô hoặc suy giảm chất lượng.

B.18 Phiếu hiệu chuẩn TDS

TDS phụ thuộc vào dung dịch chuẩn và nhiệt độ. Nếu cảm biến không bù nhiệt độ tự động, nên ghi lại nhiệt độ dung dịch khi hiệu chuẩn. Trong điều kiện làm khóa luận, có thể hiệu chuẩn ở nhiệt độ phòng và ghi rõ điều kiện đo.

Mẫu	TDS chuẩn	TDS đo	Ghi chú
Nước sạch			Dùng kiểm tra giá trị thấp.
Dung dịch chuẩn 1			Ghi nhiệt độ dung dịch.
Dung dịch chuẩn 2			Kiểm tra tuyến tính.
Dung dịch cây đang dùng			Dữ liệu vận hành thực tế.

Nếu TDS đo dao động mạnh, cần kiểm tra nguồn analog, dây tín hiệu và cách đặt đầu dò. Đầu dò TDS không nên đặt quá gần dây cấp relay hoặc nguồn

switching. Khi đo, nên chờ vài giây để giá trị ổn định rồi mới lấy mẫu trung bình.

B.19 Bộ luật điều khiển tự động đề xuất

Trong chế độ Auto, hệ thống có thể điều khiển theo luật đơn giản. Luật nên có ngưỡng bật và ngưỡng tắt khác nhau để tránh relay bật/tắt liên tục quanh một giá trị. Ví dụ, bật quạt khi CO₂ lớn hơn 1200 ppm và tắt khi CO₂ nhỏ hơn 900 ppm.

Luật	Điều kiện	Hành động
Thông gió	CO ₂ > 1200 ppm	Bật RL1 để chạy quạt hoặc quạt thông gió.
Tắt thông gió	CO ₂ < 900 ppm	Tắt RL1 nếu không còn cần thông gió.
Bổ sung sáng	Lux < 100 trong khung giờ ban ngày	Bật RL2 để cấp đèn cho cây.
Tắt đèn	Lux > 300 hoặc hết khung giờ chăm cây	Tắt RL2.
Làm mát	Nhiệt độ > 30°C và Manual tắt	Gửi IR giảm nhiệt độ điều hòa.
Cảnh báo pH	pH ngoài khoảng cấu hình	Chỉ cảnh báo, không tự điều chỉnh.

Các luật tự động cần có giới hạn tần suất điều khiển. Ví dụ, không gửi lệnh IR giảm nhiệt độ liên tục trong vài giây vì điều hòa cần thời gian phản ứng. Gateway có thể đặt thời gian chờ tối thiểu 5 đến 10 phút giữa hai lần điều khiển điều hòa tự động.

B.20 Checklist lắp đặt node trong căn hộ

Lắp đặt đúng vị trí ảnh hưởng lớn đến chất lượng dữ liệu. Node không nên đặt sát miệng gió điều hòa vì nhiệt độ đo sẽ lệch so với không gian chung. Cảm biến CO₂ nên đặt ở độ cao gần vùng hô hấp, tránh đặt sát cửa sổ hoặc quạt thổi trực tiếp. Cảm biến ánh sáng cần hướng về vùng cây nhận sáng.

Hạng mục	Kiểm tra
Vị trí node	Không đặt sát nguồn nhiệt, nước hoặc nơi dễ va chạm.
Vị trí CO2	Không đặt sát cửa sổ hoặc trực tiếp trước quạt.
Vị trí ánh sáng	Đặt gần cây để phản ánh đúng ánh sáng cây nhận được.
Dây cảm biến pH/TDS	Tránh đi song song dây relay hoặc dây nguồn công suất.
Anten LoRa	Đặt thẳng, tránh bị che bởi kim loại.
Gateway ESP32	Đặt nơi Wi-Fi ổn định và không quá xa node.
Relay	Kiểm tra tải định mức trước khi đấu thiết bị thật.
IR LED	Hướng về mắt nhận IR của điều hòa.

B.21 Checklist bảo mật triển khai

Dù hệ thống được dùng trong căn hộ, bảo mật vẫn cần được chú ý vì actuator có thể điều khiển thiết bị điện. Những điểm cần kiểm tra:

- Không để token ThingsBoard trong tài liệu công khai.
- Không dùng mật khẩu mặc định cho tài khoản app.
- Bật HTTPS nếu ThingsBoard triển khai trên server riêng.
- Phân quyền tài khoản xem dữ liệu và tài khoản điều khiển.
- Không cho phép app gửi lệnh khi chưa đăng nhập.
- Ghi log lệnh điều khiển quan trọng như bật/tắt relay.

Nếu hệ thống được mở ra Internet, cần cấu hình firewall, chứng chỉ TLS và chính sách mật khẩu mạnh. Nếu chỉ dùng trong mạng nội bộ, vẫn nên bảo vệ access token và hạn chế người dùng có quyền điều khiển actuator.

B.22 Mẫu biên bản nghiệm thu

Biên bản nghiệm thu ghi lại trạng thái cuối cùng của hệ thống sau khi hoàn thành triển khai.

Hạng mục	Kết quả	Ghi chú
Node đọc TDS	Đạt / Không đạt	
Node đọc pH	Đạt / Không đạt	
Node đọc ánh sáng	Đạt / Không đạt	
Node đọc CO2	Đạt / Không đạt	
Node đọc nhiệt độ, độ ẩm	Đạt / Không đạt	
Gateway nhận LoRa	Đạt / Không đạt	
ThingsBoard nhận telemetry	Đạt / Không đạt	
App hiển thị Sensor	Đạt / Không đạt	
App điều khiển RL1/RL2	Đạt / Không đạt	
App điều khiển điều hòa IR	Đạt / Không đạt	
Cảnh báo CO2/ánh sáng/pH	Đạt / Không đạt	

Khi tất cả hạng mục chính đạt yêu cầu, hệ thống có thể được xem là hoàn thành ở mức nguyên mẫu. Những hạng mục chưa đạt cần ghi rõ nguyên nhân và kế hoạch khắc phục để tránh bỏ sót trong quá trình hoàn thiện báo cáo.

Tài liệu tham khảo

- [1] Arduino Documentation, *Arduino Pro Mini*. <https://docs.arduino.cc/retired/boards/arduino-pro-mini/>
- [2] Espressif Systems, *ESP32 Technical Reference Manual*. <https://www.espressif.com/en/support/documents/technical-documents>
- [3] ThingsBoard Documentation, *IoT platform documentation*. <https://thingsboard.io/docs/>
- [4] Flutter Documentation, *Build apps for any screen*. <https://docs.flutter.dev/>
- [5] LoRa Alliance, *LoRaWAN for developers*. <https://lora-alliance.org/>
- [6] World Health Organization, *Air pollution and health*, 2020.
- [7] United States Environmental Protection Agency, *Indoor Air Quality Basics*, 2021.
- [8] NASA, *NASA Clean Air Study: Plants for Indoor Air Quality*, 1989.
- [9] X. Wang et al., “IoT Applications in Smart Agriculture,” *Journal of Agricultural Technology*, vol. 17, no. 3, pp. 45–58, 2021.
- [10] R. Bose et al., “IoT in Hydroponics,” *Journal of Smart Farming*, vol. 5, no. 2, pp. 123–135, 2020.
- [11] J. Liu et al., “Real-Time Monitoring of Indoor Air,” *Environmental Sensors Journal*, vol. 12, no. 4, pp. 200–210, 2019.
- [12] A. Singh et al., “IoT in Air Quality Management,” *IoT and Environment Journal*, vol. 8, no. 1, pp. 50–60, 2022.
- [13] Hydroponics Association, *Hydroponic Systems Manual*, 2023.

- [14] H. Singh and D. Bruce, *Electrical Conductivity and pH Guide for Hydroponics*, Oklahoma Cooperative Extension Service, HLA-6722, 2016.
- [15] A. Karle, P. Naik, R. Gawali, S. Deshmukh, and D. Suroshi, “Indoor Air Quality Management Using Hydroponic Systems: A Survey Paper,” *International Journal for Research in Applied Science & Engineering Technology*, vol. 13, no. VI, pp. 518–524, 2025.
- [16] Y. H. Dewir, D. Chakrabarty, M. B. Ali, E. J. Hahn, and K. Y. Paek, “Effects of hydroponic solution EC, substrates, PPF and nutrient scheduling on growth and photosynthetic competence during acclimatization of micropropagated *Spathiphyllum* plantlets,” *Plant Growth Regulation*, vol. 46, pp. 241–251, 2005.
- [17] Bright Lane Gardens, *How To Grow A Peace Lily In Water (Hydroponic Method)*, 2026.
- [18] V. Neveln, *How to Grow and Care for a Peace Lily Plant Indoors*, Better Homes & Gardens, updated Apr. 9, 2026.
- [19] O. Kucukhuseyin, “CO₂ monitoring and indoor air quality,” *REHVA Journal*, pp. 54–56, Feb. 2021.